

종합실습4_제출리포트_작성틀



종합실습 4 - E-Commerce 데이터 요청 및 SQL 튜닝 리포트

- 소속: 판교 5반
- 성명: 홍은비
- DBMS: PostgreSQL _____
- 실행환경: Local PC / DBeaver _____
- 측정 명령: `EXPLAIN (ANALYZE, BUFFERS, SUMMARY)`

시드 데이터는 `random()` 으로 생성되므로 실행 결과와 시간은 환경마다 달라질 수 있다. 절대시간뿐 아니라 Scan/Join 방식, 반복 횟수, 예상·실제 행 수, Buffers 변화를 함께 비교한다.

1. 실습 목적

구매·재무 등 다른 실무 팀에서 요청한 E-Commerce 자료를 정확한 기준으로 산출하고, 같은 결과를 더 효율적으로 반환하도록 SQL과 인덱스를 개선한다. Q1~Q10은 튜닝 전후 쿼리와 실행계획을 비교하고, 별도로 PostgreSQL의 세 가지 물리 조인과 Materialized View 활용 방법을 검토한다.

2. 공통 데이터 기준

항목	정의
실매출 주문	<code>order_status IN ('paid','shipped','delivered')</code>
주문 매출	<code>SUM(order_items.line_total)</code>
AOV	실매출 합계 ÷ 실매출 주문 수
Q1 기간	실행시점 기준 최근 1개월
Q3 기간	실행시점 기준 최근 90일
Q6 재구매	첫 구매 후 30일 이내 추가 실구매
Q10 대상	전체 3,000명 누적매출 상위 1%인 30명
Q10 표시금액	선정된 30명의 최근 60일 실매출

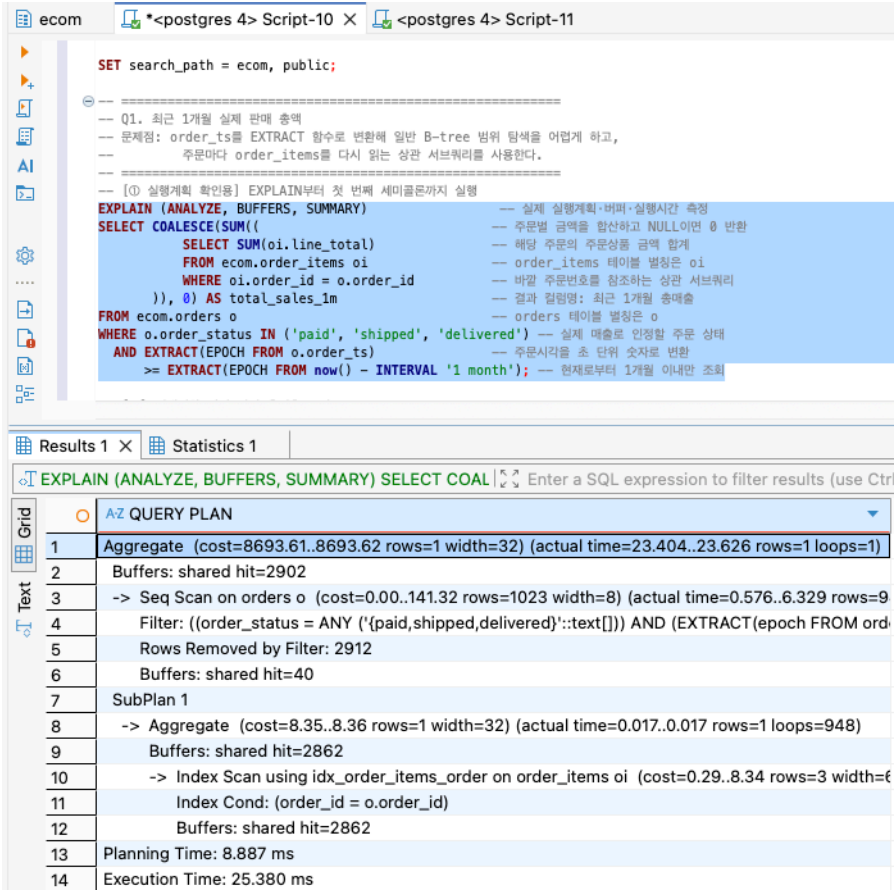
3. Q1 - 최근 1개월 실제 판매 총액

요청사항

`paid`, `shipped`, `delivered` 상태이면서 `now() - interval '1 month'` 이후인 주문의 `line_total` 합계를 제공한다.

튜닝 전 쿼리·결과 화면

Q1 튜닝 전



느린 이유

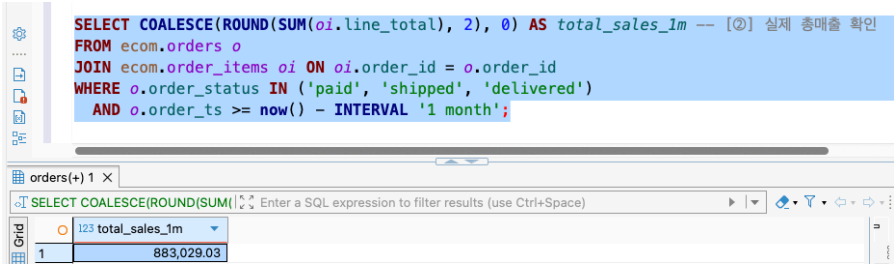
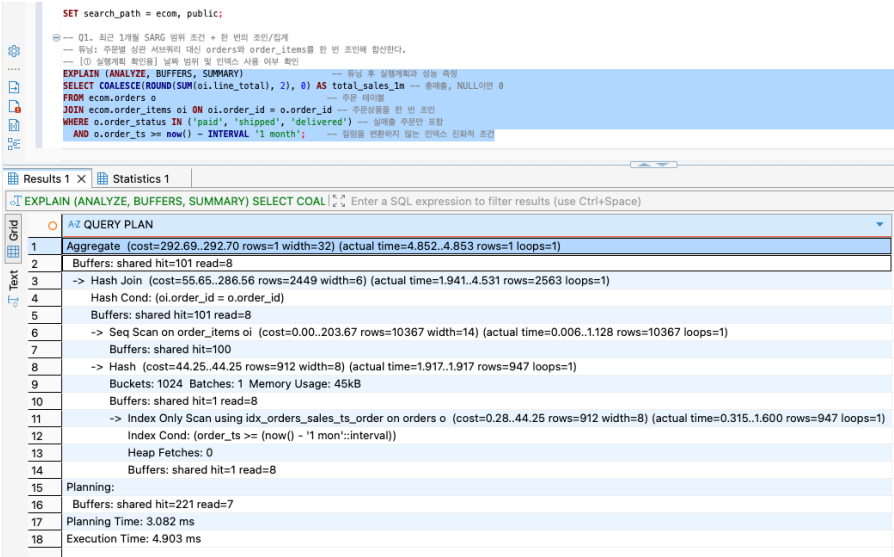
`order_ts`를 함수로 변환하면 일반 날짜 인덱스의 범위탐색이 어려워질 수 있다. 또한 주문마다 주문상품 합계를 상관 서브쿼리로 반복 조회한다.

개선 방법

날짜 컬럼을 그대로 둔 SARG 범위 조건을 사용하고, 주문과 주문상품을 한 번 조인해 전체 합계를 계산한다. 실매출 상태만 포함하는 부분 인덱스로 대상 주문 탐색 범위를 줄인다.

튜닝 후 쿼리·결과 화면

Q1 튜닝 후



구분	Scan/Join 방식	Buffers	Execution Time	결과 금액
튜닝 전	_____	_____	_____ ms	_____
튜닝 후	_____	_____	_____ ms	_____

결과 해석: _____

4. Q2 - 월별 주문 수·매출·AOV

요청사항

월별 실매출 주문 수, 총매출, 주문당 평균 금액을 제공한다.

Q2 튜닝 전

요청사항

최근 90일 실매출을 카테고리별로 집계해 상위 10개를 제공한다.

Q3 튜닝 전

Q3. 최근 90일 카테고리의 Top 10 문제점: 비-SARG 날짜 조건을 사용하고 상세 데이터를 그대로 집계한다. [O] 실행계획 확인용] 최근 90일 필터와 테이블 조건 계획 확인 실행 계획과 실행 속성 EXPLAIN (ANALYZE, BUFFERS, SUMMARY) SELECT c.category_id, c.category_name, ROUND(SUM(oi.line_total), 2) AS revenue_90d FROM ecom.orders o JOIN ecom.order_items oi ON oi.order_id = o.order_id JOIN ecom.products p ON p.product_id = oi.product_id JOIN ecom.categories c ON c.category_id = p.category_id WHERE o.order_status IN ('paid', 'shipped', 'delivered') AND o.order_ts::date >= CURRENT_DATE - 90 GROUP BY c.category_id, c.category_name ORDER BY revenue_90d DESC, c.category_id LIMIT 10;	카테고리별 최근 90일 매출 주문에서 시작 주문항을 연결 상품 연결 카테고리 연결 실매출 주문만 포함 Timestamp date로 변환해 최근 90일 필터 카테고리별로 묶어 집단 매출이 큰 순서로 정렬 상위 10개만 표시
Results 1 × Statistics 1 EXPLAIN (ANALYZE, BUFFERS, SUMMARY) SELECT c.category_id, c.category_name, ROUND(SUM(oi.line_total), 2) AS revenue_90d FROM ecom.orders o JOIN ecom.order_items oi ON oi.order_id = o.order_id JOIN ecom.products p ON p.product_id = oi.product_id JOIN ecom.categories c ON c.category_id = p.category_id WHERE o.order_status IN ('paid', 'shipped', 'delivered') AND o.order_ts::date >= CURRENT_DATE - 90 GROUP BY c.category_id, c.category_name ORDER BY revenue_90d DESC, c.category_id LIMIT 10;	Enter a SQL expression to filter results (use Ctrl+Space)
Grid Text Record	1 Limit (cost=497.38..497.41 rows=10 width=72) (actual time=11.220..11.225 rows=10 loops=1) 2 Buffers: shared hit=148 3 -> Sort (cost=497.38..500.06 rows=1070 width=72) (actual time=11.219..11.222 rows=10 loops=1) 4 Sort Key: (round(sum(oi.line_total), 2)) DESC, c.category_id 5 Sort Method: quicksort Memory: 25kB 6 Buffers: shared hit=148 7 -> HashAggregate (cost=458.21..474.26 rows=1070 width=72) (actual time=10.966..10.979 rows=10 loops=1) 8 Group Key: c.category_id 9 Batches: 1 Memory Usage: 73kB 10 Buffers: shared hit=148 11 -> Hash Join (cost=199.04..444.47 rows=2748 width=46) (actual time=3.828..9.543 rows=6662 loops=1) 12 Hash Cond: (p.category_id = c.category_id) 13 Buffers: shared hit=148 14 -> Hash Join (cost=164.96..403.15 rows=2748 width=14) (actual time=3.311..7.831 rows=6662 loops=1) 15 Hash Cond: (oi.product_id = p.product_id) 16 Buffers: shared hit=147 17 -> Hash Join (cost=144.46..375.38 rows=2748 width=14) (actual time=2.841..6.120 rows=6662 loops=1) 18 Hash Cond: (oi.order_id = o.order_id) 19 Buffers: shared hit=140 20 -> Seq Scan on order_items oi (cost=0.00..203.67 rows=10367 width=22) (actual time=0.015..1.308 rows=10367 loops=1) 21 Buffers: shared hit=100 22 -> Hash (cost=131.68..131.68 rows=1023 width=8) (actual time=2.502..2.502 rows=2483 loops=1) 23 Buckets: 4096 (originally 1024) Batches: 1 (originally 1) Memory Usage: 129kB 24 Buffers: shared hit=40

Results 1 × Statistics 1 EXPLAIN (ANALYZE, BUFFERS, SUMMARY) SELECT c.category_id, c.category_name, ROUND(SUM(oi.line_total), 2) AS revenue_90d FROM ecom.orders o JOIN ecom.order_items oi ON oi.order_id = o.order_id JOIN ecom.products p ON p.product_id = oi.product_id JOIN ecom.categories c ON c.category_id = p.category_id WHERE o.order_status IN ('paid', 'shipped', 'delivered') AND o.order_ts::date >= CURRENT_DATE - 90 GROUP BY c.category_id, c.category_name ORDER BY revenue_90d DESC, c.category_id LIMIT 10;	Enter a SQL expression to filter results (use Ctrl+Space)
Grid Text Record	20 -> Seq Scan on order_items oi (cost=0.00..203.67 rows=10367 width=22) (actual time=0.015..1.308 rows=10367 loops=1) 21 Buffers: shared hit=100 22 -> Hash (cost=131.68..131.68 rows=1023 width=8) (actual time=2.502..2.502 rows=2483 loops=1) 23 Buckets: 4096 (originally 1024) Batches: 1 (originally 1) Memory Usage: 129kB 24 Buffers: shared hit=40 25 -> Seq Scan on orders o (cost=0.00..131.68 rows=1023 width=8) (actual time=0.023..1.672 rows=2483 loops=1) 26 Filter: ((order_status = ANY ('paid,shipped,delivered'))::text[]) AND ((order_ts)::date >= (CURRENT_DATE - 90)) 27 Rows Removed by Filter: 1377 28 Buffers: shared hit=40 29 -> Hash (cost=13.00..13.00 rows=600 width=16) (actual time=0.449..0.450 rows=600 loops=1) 30 Buckets: 1024 Batches: 1 Memory Usage: 37kB 31 Buffers: shared hit=7 32 -> Seq Scan on products p (cost=0.00..13.00 rows=600 width=16) (actual time=0.006..0.368 rows=600 loops=1) 33 Buffers: shared hit=7 34 -> Hash (cost=20.70..20.70 rows=1070 width=40) (actual time=0.276..0.276 rows=14 loops=1) 35 Buckets: 2048 Batches: 1 Memory Usage: 17kB 36 Buffers: shared hit=1 37 -> Seq Scan on categories c (cost=0.00..20.70 rows=1070 width=40) (actual time=0.256..0.257 rows=14 loops=1) 38 Buffers: shared hit=1 39 Planning: 40 Buffers: shared hit=27 41 Planning Time: 6.885 ms 42 Execution Time: 11.863 ms

느린 이유와 개선 방법

비-SARG 날짜 조건은 인덱스 범위 탐색을 방해할 수 있다. 튜닝 후에는 최근 실매출 주문을 먼저 축소하고 `order_items → products → categories` 순서로 연결한다.

Q3 튜닝 후

Q3. 기간/상태로 orders를 먼저 축소하고 SARG 범위 조건 사용 튜닝: 최근 90일 실매출 주문만 먼저 골라 조건해야 할 행 수를 줄인다. [O] 실행계획 확인용] 최근 주문 선택 필터와 조건 계획 확인 EXPLAIN (ANALYZE, BUFFERS, SUMMARY) WITH recent_sales AS MATERIALIZED (SELECT order_id FROM ecom.orders WHERE order_status IN ('paid', 'shipped', 'delivered') AND order_ts >= CURRENT_DATE - 90) SELECT c.category_id, c.category_name, ROUND(SUM(oi.line_total), 2) AS revenue_90d FROM recent_sales rs JOIN ecom.order_items oi ON oi.order_id = rs.order_id JOIN ecom.products p ON p.product_id = oi.product_id JOIN ecom.categories c ON c.category_id = p.category_id GROUP BY c.category_id, c.category_name ORDER BY revenue_90d DESC, c.category_id LIMIT 10;	최근 주문을 먼저 계산해 저장하는 CTE 가림을 변환하지 않는 최근 90일 조건
Results 1 × Statistics 1 WITH recent_sales AS MATERIALIZED (SELECT order_id FROM ecom.orders WHERE order_status IN ('paid', 'shipped', 'delivered') AND order_ts >= CURRENT_DATE - 90) SELECT c.category_id, c.category_name, ROUND(SUM(oi.line_total), 2) AS revenue_90d FROM recent_sales rs JOIN ecom.order_items oi ON oi.order_id = rs.order_id JOIN ecom.products p ON p.product_id = oi.product_id JOIN ecom.categories c ON c.category_id = p.category_id GROUP BY c.category_id, c.category_name ORDER BY revenue_90d DESC, c.category_id LIMIT 10;	Enter a SQL expression to filter results (use Ctrl+Space)
Grid Text Record	1 Limit (cost=497.38..497.41 rows=10 width=72) (actual time=11.220..11.225 rows=10 loops=1) 2 Buffers: shared hit=148 3 -> Sort (cost=497.38..500.06 rows=1070 width=72) (actual time=11.219..11.222 rows=10 loops=1) 4 Sort Key: (round(sum(oi.line_total), 2)) DESC, c.category_id 5 Sort Method: quicksort Memory: 25kB 6 Buffers: shared hit=148 7 -> HashAggregate (cost=458.21..474.26 rows=1070 width=72) (actual time=10.966..10.979 rows=10 loops=1) 8 Group Key: c.category_id 9 Batches: 1 Memory Usage: 73kB 10 Buffers: shared hit=148 11 -> Hash Join (cost=199.04..444.47 rows=2748 width=46) (actual time=3.828..9.543 rows=6662 loops=1) 12 Hash Cond: (p.category_id = c.category_id) 13 Buffers: shared hit=148 14 -> Hash Join (cost=164.96..403.15 rows=2748 width=14) (actual time=3.311..7.831 rows=6662 loops=1) 15 Hash Cond: (oi.product_id = p.product_id) 16 Buffers: shared hit=147 17 -> Hash Join (cost=144.46..375.38 rows=2748 width=14) (actual time=2.841..6.120 rows=6662 loops=1) 18 Hash Cond: (oi.order_id = o.order_id) 19 Buffers: shared hit=140 20 -> Seq Scan on order_items oi (cost=0.00..203.67 rows=10367 width=22) (actual time=0.015..1.308 rows=10367 loops=1) 21 Buffers: shared hit=100 22 -> Hash (cost=131.68..131.68 rows=1023 width=8) (actual time=2.502..2.502 rows=2483 loops=1) 23 Buckets: 4096 (originally 1024) Batches: 1 (originally 1) Memory Usage: 129kB 24 Buffers: shared hit=40

SELECT c.category_id, c.category_name, ROUND(SUM(oi.line_total), 2) AS revenue_90d FROM ecom.orders o JOIN ecom.order_items oi ON oi.order_id = o.order_id JOIN ecom.products p ON p.product_id = oi.product_id JOIN ecom.categories c ON c.category_id = p.category_id WHERE o.order_status IN ('paid', 'shipped', 'delivered') AND o.order_ts::date >= CURRENT_DATE - 90 GROUP BY c.category_id, c.category_name ORDER BY revenue_90d DESC, c.category_id LIMIT 10;	[O] 실제 Top 10 결과 조회
Results 1 × Statistics 1 SELECT c.category_id, c.category_name, ROUND(SUM(oi.line_total), 2) AS revenue_90d FROM ecom.orders o JOIN ecom.order_items oi ON oi.order_id = o.order_id JOIN ecom.products p ON p.product_id = oi.product_id JOIN ecom.categories c ON c.category_id = p.category_id WHERE o.order_status IN ('paid', 'shipped', 'delivered') AND o.order_ts::date >= CURRENT_DATE - 90 GROUP BY c.category_id, c.category_name ORDER BY revenue_90d DESC, c.category_id LIMIT 10;	Enter a SQL expression to filter results (use Ctrl+Space)
Grid Text Record	categories 1 × SELECT c.category_id, c.category_name, ROUND(SUM(oi.line_total), 2) AS revenue_90d FROM ecom.orders o JOIN ecom.order_items oi ON oi.order_id = o.order_id JOIN ecom.products p ON p.product_id = oi.product_id JOIN ecom.categories c ON c.category_id = p.category_id WHERE o.order_status IN ('paid', 'shipped', 'delivered') AND o.order_ts::date >= CURRENT_DATE - 90 GROUP BY c.category_id, c.category_name ORDER BY revenue_90d DESC, c.category_id LIMIT 10;
123 category_id 13 Outdoor 2 Phones 3 6 Appliances 4 14 Fitness 5 11 Shoes 6 7 Cookware 7 4 Audio 8 3 Laptops 9 9 Men 10 10 Women	123 revenue_90d 758,672.65 541,480.74 138,788.46 125,711.82 124,829.38 117,640.32 112,653.77 109,500.6 99,046.69 96,304.36

Results 1 X	
EXPLAIN (ANALYZE, BUFFERS, SUMMARY) WITH recent	
Grid	
1 Limit (cost=659.69..659.71 rows=10 width=72) (actual time=11.383..11.387 rows=10 loops=1)	
2 Buffers: shared hit=120 read=9	
3 CTE recent_sales	
4 -> Index Only Scan using idx_orders_sales_ts_order on orders (cost=0.28..111.02 rows=2442 width=8) (actual time=0.413..1.477 rows=10 loops=1)	
5 Index Cond: (order_ts >= (CURRENT_DATE - 90))	
6 Heap Fetches: 0	
7 Buffers: shared hit=12 read=9	
8 -> Sort (cost=548.67..551.34 rows=1070 width=72) (actual time=11.382..11.385 rows=10 loops=1)	
9 Sort Key: (round(sum(oi.line_total), 2)) DESC, c.category_id	
10 Sort Method: quicksort Memory: 25kB	
11 Buffers: shared hit=120 read=9	
12 -> HashAggregate (cost=509.50..525.55 rows=1070 width=72) (actual time=11.354..11.369 rows=10 loops=1)	
13 Group Key: c.category_id	
14 Batches: 1 Memory Usage: 73kB	
15 Buffers: shared hit=120 read=9	
16 -> Hash Join (cost=133.94..476.70 rows=6559 width=46) (actual time=2.623..9.671 rows=6662 loops=1)	
17 Hash Cond: (p.category_id = c.category_id)	
18 Buffers: shared hit=120 read=9	
19 -> Hash Join (cost=99.87..425.34 rows=6559 width=14) (actual time=2.587..8.147 rows=6662 loops=1)	
20 Hash Cond: (oi.product_id = p.product_id)	
21 Buffers: shared hit=119 read=9	
22 -> Hash Join (cost=79.37..387.50 rows=6559 width=14) (actual time=2.372..6.349 rows=6662 loops=1)	
23 Hash Cond: (oi.order_id = rs.order_id)	
24 Buffers: shared hit=112 read=9	
25 -> Seq Scan on order_items oi (cost=0.00..203.67 rows=10367 width=22) (actual time=0.016..1.415 rows=10367 loops=1)	
26 Buffers: shared hit=100	
27 -> Hash (cost=48.84..48.84 rows=2442 width=8) (actual time=2.335..2.335 rows=2483 loops=1)	
28 Buckets: 4096 Batches: 1 Memory Usage: 129kB	
29 Buffers: shared hit=12 read=9	
30 -> CTE Scan on recent_sales rs (cost=0.00..48.84 rows=2442 width=8) (actual time=0.414..2.010 rows=2483 loops=1)	
31 Buffers: shared hit=12 read=9	
32 -> Hash (cost=13.00..13.00 rows=600 width=16) (actual time=0.194..0.194 rows=600 loops=1)	
33 Buckets: 1024 Batches: 1 Memory Usage: 37kB	
34 Buffers: shared hit=7	
35 -> Seq Scan on products p (cost=0.00..13.00 rows=600 width=16) (actual time=0.008..0.101 rows=600 loops=1)	
36 Buffers: shared hit=7	
37 -> Hash (cost=20.70..20.70 rows=1070 width=40) (actual time=0.019..0.019 rows=14 loops=1)	
38 Buckets: 2048 Batches: 1 Memory Usage: 17kB	
39 Buffers: shared hit=1	
40 -> Seq Scan on categories c (cost=0.00..20.70 rows=1070 width=40) (actual time=0.008..0.010 rows=14 loops=1)	
41 Buffers: shared hit=1	
Planning:	
43 Buffers: shared hit=64	
44 Planning Time: 2.458 ms	
45 Execution Time: 11.534 ms	

구분	Orders 접근 방식	Join 방식	Buffers	Execution Time
튜닝 전	_____	_____	_____	_____ ms
튜닝 후	_____	_____	_____	_____ ms

Top 10 결과 및 해석: _____

6. Q4 - 제품별 누적매출 RANK Top 20

요청사항

제품별 누적매출을 먼저 계산하고 **RANK()** 로 상위 20개 제품을 제공한다.

Q4 튜닝 전

ecom	
*<postgres 4> Script-10 X	
<postgres 4> Script-11	
-- Q4. 제품별 누적매출 RANK() Top 20	
-- 문제점: 600개 제품 각각에 대해 매출 상한 서브쿼리를 실행한다.	
-- [Q] 실행계획 확인용] 번째 SubPlan의 Loops와 실행시간 확인	
EXPLAIN (ANALYZE, BUFFERS, SUMMARY)	
WITH product_revenue AS (	
SELECT p.product_id,	
p.product_name,	
COALESCE((	
SELECT SUM(oi.line_total)	
FROM ecom.order_items oi	
JOIN ecom.orders o ON o.order_id = oi.order_id	
WHERE oi.product_id = p.product_id	
AND o.order_status IN ('paid', 'shipped', 'delivered')	
), 0) AS revenue	
FROM ecom.products p	
), ranked AS (	
SELECT product_id,	
product_name,	
revenue,	
RANK() OVER (ORDER BY revenue DESC) AS revenue_rank	
FROM product_revenue	
WHERE revenue > 0	
SELECT product_id, product_name, ROUND(revenue, 2) AS revenue, revenue_rank	
FROM ranked	
WHERE revenue_rank <= 20	
ORDER BY revenue_rank, product_id;	
Results 1 X	
EXPLAIN (ANALYZE, BUFFERS, SUMMARY) WITH product	
Grid	
1 Sort (cost=158635.79..158636.29 rows=200 width=59) (actual time=36.685..36.687 rows=20 loops=1)	
2 Sort Key: ranked.revenue_rank, ranked.product_id	
3 Sort Method: quicksort Memory: 26kB	
4 Buffers: shared hit=70677	
5 -> Subquery Scan on ranked (cost=126903.76..158628.14 rows=200 width=59) (actual time=36.334..36.354 rows=20 loops=1)	
6 Buffers: shared hit=70677	
7 -> WindowAgg (cost=126903.76..158627.64 rows=200 width=59) (actual time=36.332..36.349 rows=20 loops=1)	
8 Run Condition: (rank() OVER (?) <= 20)	
9 Buffers: shared hit=70677	
10 -> Sort (cost=126903.74..126904.24 rows=200 width=51) (actual time=36.305..36.308 rows=21 loops=1)	
11 Sort Key: (COALESCE((SubPlan 1), '0'::numeric)) DESC	
12 Sort Method: quicksort Memory: 52kB	
13 Buffers: shared hit=70677	
14 -> Seq Scan on products p (cost=0.00..126896.10 rows=200 width=51) (actual time=1.089..35.299 rows=585 loops=1)	
15 Filter: (COALESCE((SubPlan 2), '0'::numeric) > '0'::numeric)	
16 Rows Removed by Filter: 15	

WITH product_revenue AS (	
SELECT p.product_id,	
p.product_name,	
COALESCE((	
SELECT SUM(oi.line_total)	
FROM ecom.order_items oi	
JOIN ecom.orders o ON o.order_id = oi.order_id	
WHERE oi.product_id = p.product_id	
AND o.order_status IN ('paid', 'shipped', 'delivered')	
), 0) AS revenue	
FROM ecom.products p	
), ranked AS (	
SELECT product_id, product_name, revenue,	
RANK() OVER (ORDER BY revenue DESC) AS revenue_rank	
FROM product_revenue	
WHERE revenue > 0	
SELECT product_id, product_name, ROUND(revenue, 2) AS revenue, revenue_rank	
FROM ranked	
WHERE revenue_rank <= 20	
ORDER BY revenue_rank, product_id;	
products 1 X	
WITH product_revenue AS (SELECT p.product_id, p.prod	
Grid	
123 product_id	
A product_name	
123 revenue	
123 revenue_rank	
1 293 Product 465 859,785.6 1	
2 455 Product 378 430,633.92 2	
3 501 Product 439 9,189.04 3	
4 272 Product 315 8,234.57 4	
5 264 Product 225 8,109.2 5	
6 528 Product 549 8,040.23 6	
7 315 Product 136 7,868.31 7	
8 69 Product 42 7,817.88 8	
9 235 Product 114 7,584.68 9	
10 533 Product 39 7,426.7 10	
11 236 Product 584 7,398.53 11	
12 63 Product 32 7,375.03 12	
13 55 Product 481 6,944.38 13	
14 137 Product 463 6,919.99 14	
15 33 Product 211 6,859.89 15	
16 449 Product 418 6,748.8 16	
17 311 Product 96 6,699.06 17	
18 181 Product 264 6,586.17 18	
19 429 Product 498 6,539.14 19	
20 81 Product 522 6,463.1 20	

Results 1 X	
EXPLAIN (ANALYZE, BUFFERS, SUMMARY) WITH product Enter a SQL expression to filter results (use Ctrl+Space)	
Grid	AZ QUERY PLAN
Text	13 Buffers: shared hit=70677
	14 -> Seq Scan on products p (cost=0.00..126896.10 rows=200 width=51) (actual time=1.089..35.299 rows=585 loops=1)
	15 Filter: (COALESCE((SubPlan 2), '0')::numeric) > '0'::numeric)
	16 Rows Removed by Filter: 15
	17 Buffers: shared hit=70677
	18 SubPlan 1
	19 -> Aggregate (cost=158.59..158.60 rows=1 width=32) (actual time=0.025..0.025 rows=1 loops=585)
	20 Buffers: shared hit=35254
	21 -> Nested Loop (cost=4.70..158.55 rows=14 width=6) (actual time=0.006..0.024 rows=14 loops=585)
	22 Buffers: shared hit=35254
	23 -> Bitmap Heap Scan on order_items oi (cost=4.42..49.43 rows=17 width=14) (actual time=0.002..0.005 rows=17 loops=585)
	24 Recheck Cond: (product_id = p.product_id)
	25 Heap Blocks: exact=3099
	26 Buffers: shared hit=4270
	27 -> Bitmap Index Scan on idx_order_items_product (cost=0.00..4.41 rows=17 width=0) (actual time=0.000..0.001 rows=17 loops=585)
	28 Index Cond: (product_id = p.product_id)
	29 Buffers: shared hit=1171
	30 -> Index Scan using orders_pkey on orders o (cost=0.28..6.42 rows=1 width=8) (actual time=0.001..0.001 rows=1 loops=585)
	31 Index Cond: (order_id = oi.order_id)
	32 Filter: (order_status = ANY ('(paid,shipped,delivered')::text[]))
	33 Rows Removed by Filter: 0
	34 Buffers: shared hit=30984
	35 SubPlan 2
	36 -> Aggregate (cost=158.59..158.60 rows=1 width=32) (actual time=0.033..0.033 rows=1 loops=600)
	37 Buffers: shared hit=35416
	38 -> Nested Loop (cost=4.70..158.55 rows=14 width=6) (actual time=0.010..0.031 rows=14 loops=600)
	39 Buffers: shared hit=35416
	40 -> Bitmap Heap Scan on order_items oi_1 (cost=4.42..49.43 rows=17 width=14) (actual time=0.005..0.009 rows=17 loops=600)
	41 Recheck Cond: (product_id = p.product_id)
	42 Heap Blocks: exact=3114
	43 Buffers: shared hit=4315
	44 -> Bitmap Index Scan on idx_order_items_product (cost=0.00..4.41 rows=17 width=0) (actual time=0.000..0.001 rows=17 loops=600)
	45 Index Cond: (product_id = p.product_id)
	46 Buffers: shared hit=1201
	47 -> Index Scan using orders_pkey on orders o_1 (cost=0.28..6.42 rows=1 width=8) (actual time=0.001..0.001 rows=1 loops=600)
	48 Index Cond: (order_id = oi_1.order_id)
	49 Filter: (order_status = ANY ('(paid,shipped,delivered')::text[]))
	50 Rows Removed by Filter: 0
	51 Buffers: shared hit=31101
Record	52 Planning:
	53 Buffers: shared hit=24
	54 Planning Time: 1.451 ms
	55 Execution Time: 37.373 ms

느린 이유와 개선 방법

튜닝 전에는 제품마다 주문상품과 주문을 반복 조회한다. 튜닝 후에는 전체 매출을 제품별로 한 번 집계한 후 Window Function을 적용한다.

Q4 튜닝 후

Q4. 전체 매출을 한 번만 집계한 뒤 RANK 적용	
튜닝: 제품마다 매출을 다시 찾기 않고 전체 데이터를 한 번 읽어 제품별로 묶는다.	
[O 실행계획 확인용] 반복 SubPlan이 제거됐는지 확인	
EXPLAIN (ANALYZE, BUFFERS, SUMMARY)	
WITH product_revenue AS (-- 전체 데이터를 제품별로 한 번 집계	
SELECT p.product_id,	
p.product_name,	
SUM(oi.line_total) AS revenue	
FROM ecom.orders o	
JOIN ecom.order_items oi ON oi.order_id = o.order_id	
JOIN ecom.products p ON p.product_id = oi.product_id	
WHERE o.order_status IN ('paid', 'shipped', 'delivered')	
GROUP BY p.product_id, p.product_name	
), ranked AS (-- 집계 결과에 순위를 부여	
SELECT product_id, product_name, revenue,	
RANK() OVER (ORDER BY revenue DESC) AS revenue_rank -- 매출 내림차순 공동 순위	
FROM product_revenue	
SELECT product_id, product_name, ROUND(revenue, 2) AS revenue, revenue_rank	
FROM ranked	
WHERE revenue_rank <= 20	
ORDER BY revenue_rank, product_id;	

Results 1 X	
EXPLAIN (ANALYZE, BUFFERS, SUMMARY) WITH product Enter a SQL expression to filter results (use Ctrl+Space)	
Grid	AZ QUERY PLAN
Text	1 Sort (cost=520.70..522.20 rows=600 width=59) (actual time=9.509..9.513 rows=20 loops=1)
	2 Sort Key: ranked.revenue_rank, ranked.product_id
	3 Sort Method: quicksort Memory: 26kB
	4 Buffers: shared hit=147
	5 -> Subquery Scan on ranked (cost=481.03..493.01 rows=600 width=59) (actual time=9.481..9.501 rows=20 loops=1)
	6 Buffers: shared hit=147
	7 -> WindowAgg (cost=481.03..491.51 rows=600 width=59) (actual time=9.479..9.495 rows=20 loops=1)
	8 Run Condition: (rank() OVER (?) <= 20)
	9 Buffers: shared hit=147
	10 -> Sort (cost=481.01..482.51 rows=600 width=51) (actual time=9.473..9.477 rows=21 loops=1)
	11 Sort Key: product_revenue.revenue DESC
	12 Sort Method: quicksort Memory: 52kB
	13 Buffers: shared hit=147
	14 -> Subquery Scan on product_revenue (cost=445.83..453.33 rows=600 width=51) (actual time=9.089..9.292 rows=585 loops=1)
	15 Buffers: shared hit=147
	16 -> HashAggregate (cost=445.83..453.33 rows=600 width=51) (actual time=9.088..9.241 rows=585 loops=1)
	17 Group Key: p.product_id
	18 Batches: 1 Memory Usage: 297kB
	19 Buffers: shared hit=147
	20 -> Hash Join (cost=151.93..404.63 rows=8240 width=25) (actual time=2.183..7.128 rows=8230 loops=1)
	21 Hash Cond: (oi.product_id = p.product_id)
	22 Buffers: shared hit=147
	23 -> Hash Join (cost=131.43..362.34 rows=8240 width=14) (actual time=1.843..5.248 rows=8230 loops=1)
	24 Hash Cond: (oi.order_id = o.order_id)
	25 Buffers: shared hit=140
	26 -> Seq Scan on order_items oi (cost=0.00..203.67 rows=10367 width=22) (actual time=0.010..1.255 rows=10367 loops=1)
	27 Buffers: shared hit=100
	28 -> Hash (cost=93.08..93.08 rows=3068 width=8) (actual time=1.801..1.802 rows=3068 loops=1)
	29 Buckets: 4096 Batches: 1 Memory Usage: 152kB
	30 Buffers: shared hit=40
	31 -> Seq Scan on orders o (cost=0.00..93.08 rows=3068 width=8) (actual time=0.012..1.281 rows=3068 loops=1)
	32 Filter: (order_status = ANY ('(paid,shipped,delivered')::text[]))
	33 Rows Removed by Filter: 792
	34 Buffers: shared hit=40
	35 -> Hash (cost=13.00..13.00 rows=600 width=19) (actual time=0.315..0.316 rows=600 loops=1)
	36 Buckets: 1024 Batches: 1 Memory Usage: 39kB
	37 Buffers: shared hit=7
	38 -> Seq Scan on products p (cost=0.00..13.00 rows=600 width=19) (actual time=0.014..0.147 rows=600 loops=1)
	39 Buffers: shared hit=7
Record	40 Planning:
	41 Buffers: shared hit=114 read=3 dirtied=1
	42 Planning Time: 6.071 ms
	43 Execution Time: 9.654 ms

WITH product_revenue AS (-- [O] 실제 제품별 순위 조회	
SELECT p.product_id, p.product_name, SUM(oi.line_total) AS revenue	
FROM ecom.orders o	
JOIN ecom.order_items oi ON oi.order_id = o.order_id	
JOIN ecom.products p ON p.product_id = oi.product_id	
WHERE o.order_status IN ('paid', 'shipped', 'delivered')	
GROUP BY p.product_id, p.product_name	
), ranked AS (	
SELECT product_id, product_name, revenue,	
RANK() OVER (ORDER BY revenue DESC) AS revenue_rank	
FROM product_revenue	
SELECT product_id, product_name, ROUND(revenue, 2) AS revenue, revenue_rank	
FROM ranked	
WHERE revenue_rank <= 20	
ORDER BY revenue_rank, product_id;	

products 1 X	
WITH product_revenue AS (SELECT p.product_id, p.prod Enter a SQL expression to filter results (use Ctrl+Space)	
Grid	123 product_id
Text	AZ product_name
	123 revenue
	123 revenue_rank
	1 293 Product 465 859,785.6 1
	2 455 Product 378 430,633.92 2
	3 501 Product 439 9,189.04 3
	4 272 Product 315 8,234.57 4
	5 264 Product 225 8,109.2 5
	6 528 Product 549 8,040.23 6
	7 315 Product 136 7,868.31 7
	8 69 Product 42 7,817.88 8
	9 235 Product 114 7,584.68 9
	10 533 Product 39 7,426.7 10
	11 236 Product 584 7,398.53 11
	12 63 Product 32 7,375.03 12
	13 55 Product 481 6,944.38 13
	14 137 Product 463 6,919.99 14
	15 33 Product 211 6,859.89 15
	16 449 Product 418 6,748.8 16
	17 311 Product 96 6,699.06 17
	18 181 Product 264 6,586.17 18
	19 429 Product 498 6,539.14 19
	20 81 Product 522 6,463.1 20

구분	반복 SubPlan	Buffers	Execution Time	1위 제품/매출
튜닝 전	_____	_____	_____ ms	_____
튜닝 후	_____	_____	_____ ms	_____

결과 해석: _____

7. Q5 - 고객별 RFM 분석

요청사항

고객별 최근성(최근 구매일), 빈도(주문 수), 금액(총매출)을 제공한다.

Q5 튜닝 전

```
-- Q5. 고객별 RFM
-- 문제점: 고객마다 최근 구매·빈도·금액을 각각 다시 조회한다.
-- 화면에는 M(금액) 상위 20행만 표시한다.
-- 의미: Recency=최근 구매 후 경과일, Frequency=구매 빈도, Monetary=구매 금액이다.
--
-- (1) 실행계획 확인용] 고객별 반복 서브쿼리와 loops 확인
EXPLAIN (ANALYZE, BUFFERS, SUMMARY) -- 고객별 반복 서브쿼리 비용 측정
SELECT c.customer_id,
       c.full_name,
       CURRENT_DATE - (
         SELECT MAX(o.order_ts)::date
         FROM ecom.orders o
         WHERE o.customer_id = c.customer_id
         AND o.order_status IN ('paid', 'shipped', 'delivered')
       ) AS recency_days, -- R: 최근 구매 후 경과일
       (
         SELECT COUNT(o.order_id)
         FROM ecom.orders o
         WHERE o.customer_id = c.customer_id
         AND o.order_status IN ('paid', 'shipped', 'delivered')
       ) AS frequency, -- F: 구매 빈도
       (
         SELECT COALESCE(SUM(oi.line_total), 0) -- 고객의 주문상품 총매출
         FROM ecom.orders o
         JOIN ecom.order_items oi ON oi.order_id = o.order_id
         WHERE o.customer_id = c.customer_id
         AND o.order_status IN ('paid', 'shipped', 'delivered')
       ) AS monetary -- M: 구매 금액
FROM ecom.customers c
WHERE EXISTS (
  SELECT 1
  FROM ecom.orders o
  WHERE o.customer_id = c.customer_id
  AND o.order_status IN ('paid', 'shipped', 'delivered')
)
ORDER BY monetary DESC, c.customer_id
LIMIT 20;
```

QUERY PLAN	
1	Limit (cost=50416.78..51093.45 rows=20 width=65) (actual time=21.113..21.351 rows=20 loops=1)
2	Buffers: shared hit=17287
3	-> Result (cost=50416.78..150418.03 rows=3000 width=65) (actual time=21.113..21.348 rows=20 loops=1)
4	Buffers: shared hit=17287
5	-> Sort (cost=50416.78..50424.28 rows=3000 width=53) (actual time=20.749..20.753 rows=20 loops=1)
6	Sort Key: (SubPlan 4) DESC, c.customer_id
7	Sort Method: top-N heapsort Memory: 26kB
8	Buffers: shared hit=16776
9	-> Hash Join (cost=159.45..50336.95 rows=3000 width=53) (actual time=2.743..20.024 rows=2217 loops=1)
10	Hash Cond: (c.customer_id = o.customer_id)
11	Buffers: shared hit=16776
12	-> Seq Scan on customers c (cost=0.00..70.00 rows=3000 width=21) (actual time=0.033..4.299 rows=3000 loops=1)
13	Buffers: shared hit=40
14	-> Hash (cost=126.84..126.84 rows=2609 width=8) (actual time=2.045..2.046 rows=2217 loops=1)
15	Buckets: 4096 Batches: 1 Memory Usage: 119kB
16	Buffers: shared hit=40
17	-> HashAggregate (cost=100.75..126.84 rows=2609 width=8) (actual time=1.343..1.516 rows=2217 loops=1)
18	Group Key: o.customer_id
19	Batches: 1 Memory Usage: 24kB
20	Buffers: shared hit=40
21	-> Seq Scan on orders o (cost=0.00..93.08 rows=3068 width=8) (actual time=0.012..0.811 rows=3068 loops=1)
22	Filter: (order_status = ANY ('paid,shipped,delivered')::text[])
23	Rows Removed by Filter: 792
24	Buffers: shared hit=40
25	SubPlan 4
26	-> Aggregate (cost=16.68..16.69 rows=1 width=32) (actual time=0.006..0.006 rows=1 loops=2217)
27	Buffers: shared hit=16696
28	-> Nested Loop (cost=0.57..16.67 rows=3 width=6) (actual time=0.003..0.005 rows=4 loops=2217)
29	Buffers: shared hit=16696
30	-> Index Scan using idx_orders_customer_ts on orders o_3 (cost=0.28..8.30 rows=1 width=8) (actual time=0.002..0.002 rows=1 loops=2217)
31	Index Cond: (customer_id = c.customer_id)
32	Filter: (order_status = ANY ('paid,shipped,delivered')::text[])
33	Rows Removed by Filter: 0
34	Buffers: shared hit=7438
35	-> Index Scan using idx_order_items_order on order_items oi (cost=0.29..8.34 rows=3 width=14) (actual time=0.001..0.002 rows=3 loops=3068)
36	Index Cond: (order_id = o_3_order_id)
37	Buffers: shared hit=9258

38	SubPlan 2
39	-> Result (cost=8.30..8.31 rows=1 width=4) (actual time=0.003..0.003 rows=1 loops=20)
40	Buffers: shared hit=60
41	InitPlan 1
42	-> Limit (cost=0.28..8.30 rows=1 width=8) (actual time=0.003..0.003 rows=1 loops=20)
43	Buffers: shared hit=60
44	-> Index Scan using idx_orders_customer_ts on orders o_1 (cost=0.28..8.30 rows=1 width=8) (actual time=0.002..0.002 rows=1 loops=20)
45	Index Cond: (customer_id = c.customer_id)
46	Filter: (order_status = ANY ('paid,shipped,delivered')::text[])
47	Buffers: shared hit=60
48	SubPlan 3
49	-> Aggregate (cost=8.30..8.31 rows=1 width=8) (actual time=0.025..0.025 rows=1 loops=20)
50	Buffers: shared hit=451
51	-> Index Scan using idx_orders_customer_ts on orders o_2 (cost=0.28..8.30 rows=1 width=8) (actual time=0.016..0.023 rows=23 loops=20)
52	Index Cond: (customer_id = c.customer_id)
53	Filter: (order_status = ANY ('paid,shipped,delivered')::text[])
54	Rows Removed by Filter: 0
55	Buffers: shared hit=451
56	Planning:
57	Buffers: shared hit=27
58	Planning Time: 3.302 ms
59	Execution Time: 21.915 ms

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

SQL

<

느린 이유와 개선 방법

튜닝 전에는 고객마다 최근 구매·빈도·금액을 각각 조회한다. 튜닝 후에는 주문별 금액을 선집계하고 고객별 MAX, COUNT, SUM 을 한 번에 계산한다.

Q5 튜닝 후

```
-- Q5. 주문별 매출 선집계 후 고객별 한 번의 GROUP BY로 RFM 산출
-- 튜닝: 고객 한 명마다 세 가지 서브쿼리를 반복하는 작업을 제거한다.
-- [① 실행계획 확인용] 고객별 반복 조회 대신 한 번에 집계하는지 확인
EXPLAIN (ANALYZE, BUFFERS, SUMMARY)
WITH order_totals AS (
    SELECT o.order_id, o.customer_id, o.order_ts,
           SUM(oi.line_total) AS order_total
    FROM ecom.orders o
    JOIN ecom.order_items oi ON oi.order_id = o.order_id
    WHERE o.order_status IN ('paid', 'shipped', 'delivered')
    GROUP BY o.order_id, o.customer_id, o.order_ts
), rfm AS (
    SELECT customer_id,
           CURRENT_DATE - MAX(order_ts)::date AS recency_days, -- 최근 구매 경과일
           COUNT(order_id) AS frequency, -- 구매 주문번호 수
           SUM(order_total) AS monetary -- 고객 총매출
    FROM order_totals
    GROUP BY customer_id
)
SELECT c.customer_id, c.full_name,
       r.recency_days, r.frequency, ROUND(r.monetary, 2) AS monetary
FROM rfm r
JOIN ecom.customers c ON c.customer_id = r.customer_id
ORDER BY r.monetary DESC, c.customer_id
LIMIT 20;
```

```
-- [②] 실제 고객별 RFM 조회
WITH order_totals AS (
    SELECT o.order_id, o.customer_id, o.order_ts,
           SUM(oi.line_total) AS order_total
    FROM ecom.orders o
    JOIN ecom.order_items oi ON oi.order_id = o.order_id
    WHERE o.order_status IN ('paid', 'shipped', 'delivered')
    GROUP BY o.order_id, o.customer_id, o.order_ts
), rfm AS (
    SELECT customer_id, CURRENT_DATE - MAX(order_ts)::date AS recency_days,
           COUNT(order_id) AS frequency, SUM(order_total) AS monetary
    FROM order_totals
    GROUP BY customer_id
)
SELECT c.customer_id, c.full_name,
       r.recency_days, r.frequency, ROUND(r.monetary, 2) AS monetary
FROM rfm r
JOIN ecom.customers c ON c.customer_id = r.customer_id
ORDER BY r.monetary DESC, c.customer_id
LIMIT 20;
```

customers 1 X						
WITH order_totals AS (SELECT o.order_id, o.customer_id, o.order_ts, SUM(oi.line_total) AS order_total FROM ecom.orders o JOIN ecom.order_items oi ON oi.order_id = o.order_id WHERE o.order_status IN ('paid', 'shipped', 'delivered') GROUP BY o.order_id, o.customer_id, o.order_ts), rfm AS (SELECT customer_id, CURRENT_DATE - MAX(order_ts)::date AS recency_days, COUNT(order_id) AS frequency, SUM(order_total) AS monetary FROM order_totals GROUP BY customer_id) SELECT c.customer_id, c.full_name, r.recency_days, r.frequency, ROUND(r.monetary, 2) AS monetary FROM rfm r JOIN ecom.customers c ON c.customer_id = r.customer_id ORDER BY r.monetary DESC, c.customer_id LIMIT 20;						
	123 customer_id	AZ full_name	123 recency_days	123 frequency	123 monetary	
1	8	Customer 8	2	23	35,154.22	
2	16	Customer 16	1	23	31,003.24	
3	18	Customer 18	3	23	30,283.04	
4	14	Customer 14	6	23	29,322.59	
5	6	Customer 6	0	22	28,703.23	
6	13	Customer 13	2	22	28,633.41	
7	2	Customer 2	8	23	28,623.31	
8	1	Customer 1	3	23	28,546.26	
9	20	Customer 20	0	22	28,531.13	
10	9	Customer 9	1	23	28,488.91	
11	17	Customer 17	8	23	27,317.53	
12	24	Customer 24	0	22	27,075.74	
13	23	Customer 23	4	22	25,717.57	
14	30	Customer 30	0	23	24,815.39	
15	4	Customer 4	1	23	24,726.24	
16	12	Customer 12	4	23	22,419.91	
17	19	Customer 19	0	23	22,412.27	
18	22	Customer 22	1	23	22,206.5	
19	7	Customer 7	1	23	22,034.39	
20	29	Customer 29	0	22	21,696.95	

Results 1 X	
EXPLAIN (ANALYZE, BUFFERS, SUMMARY) WITH order_totals AS (SELECT o.order_id, o.customer_id, o.order_ts, SUM(oi.line_total) AS order_total FROM ecom.orders o JOIN ecom.order_items oi ON oi.order_id = o.order_id WHERE o.order_status IN ('paid', 'shipped', 'delivered') GROUP BY o.order_id, o.customer_id, o.order_ts), rfm AS (SELECT customer_id, CURRENT_DATE - MAX(order_ts)::date AS recency_days, COUNT(order_id) AS frequency, SUM(order_total) AS monetary FROM order_totals GROUP BY customer_id) SELECT c.customer_id, c.full_name, r.recency_days, r.frequency, ROUND(r.monetary, 2) AS monetary FROM rfm r JOIN ecom.customers c ON c.customer_id = r.customer_id ORDER BY r.monetary DESC, c.customer_id LIMIT 20;	
1	Limit (cost=595.46..595.51 rows=20 width=97) (actual time=13.181..13.185 rows=20 loops=1)
2	Buffers: shared hit=180
3	-> Sort (cost=595.46..595.96 rows=200 width=97) (actual time=13.180..13.183 rows=20 loops=1)
4	Sort Key: r.monetary DESC, c.customer_id
5	Sort Method: top-N heapsort Memory: 28kB
6	Buffers: shared hit=180
7	-> Hash Join (cost=511.75..590.14 rows=200 width=97) (actual time=11.699..12.710 rows=2217 loops=1)
8	Hash Cond: (c.customer_id = r.customer_id)
9	Buffers: shared hit=180
10	-> Seq Scan on customers c (cost=0.00..70.00 rows=3000 width=21) (actual time=0.010..0.324 rows=3000 loops=1)
11	Buffers: shared hit=40
12	-> Hash (cost=509.25..509.25 rows=200 width=52) (actual time=11.675..11.677 rows=2217 loops=1)
13	Buckets: 4096 (originally 1024) Batches: 1 (originally 1) Memory Usage: 171kB
14	Buffers: shared hit=140
15	-> Subquery Scan on r (cost=503.25..509.25 rows=200 width=52) (actual time=10.032..11.245 rows=2217 loops=1)
16	Buffers: shared hit=140
17	-> HashAggregate (cost=503.25..507.25 rows=200 width=52) (actual time=10.031..11.010 rows=2217 loops=1)
18	Group Key: o.customer_id
19	Batches: 1 Memory Usage: 1153kB
20	Buffers: shared hit=140
21	-> HashAggregate (cost=403.54..441.89 rows=3068 width=56) (actual time=7.888..8.841 rows=3068 loops=1)
22	Group Key: o.order_id
23	Batches: 1 Memory Usage: 1393kB
24	Buffers: shared hit=140
25	-> Hash Join (cost=131.43..362.34 rows=8240 width=30) (actual time=1.422..5.116 rows=8230 loops=1)
26	Hash Cond: (oi.order_id = o.order_id)
27	Buffers: shared hit=140
28	-> Seq Scan on order_items oi (cost=0.00..203.67 rows=10367 width=14) (actual time=0.013..1.160 rows=10367 loops=1)
29	Buffers: shared hit=100
30	-> Hash (cost=93.08..93.08 rows=3068 width=24) (actual time=1.389..1.389 rows=3068 loops=1)
31	Buckets: 4096 Batches: 1 Memory Usage: 200kB
32	Buffers: shared hit=40
33	-> Seq Scan on orders o (cost=0.00..93.08 rows=3068 width=24) (actual time=0.016..0.940 rows=3068 loops=1)
34	Filter: (order_status = ANY ('{paid,shipped,delivered}':text[]))
35	Rows Removed by Filter: 792
36	Buffers: shared hit=40
37	Planning:
38	Buffers: shared hit=18
39	Planning Time: 0.729 ms
40	Execution Time: 13.785 ms

구분	집계 방식	Buffers	Execution Time	전후 결과 동일 여부
튜닝 전	고객별 상관 서브쿼리	_____	_____ ms	-
튜닝 후	주문 선집계 후 고객 GROUP BY	_____	_____ ms	_____

RFM 결과 해석: _____

8. Q6 - 첫 구매 후 30일 내 재구매율

요청사항

실구매 고객 중 첫 구매 후 30일 안에 추가 실구매가 있었던 고객의 비율을 제공한다.

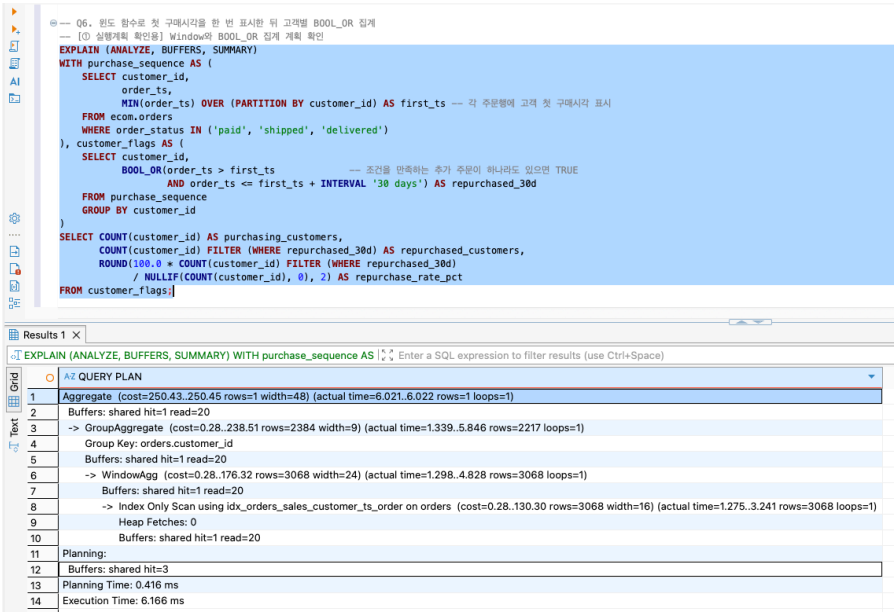
Q6 튜닝 전



느린 이유와 개선 방법

튜닝 전에는 고객별 첫 구매 이후 주문을 **EXISTS** 로 반복 탐색한다. 튜닝 후에는 Window Function으로 첫 구매시각을 표시하고 고객별 **BOOL_OR** 로 재구매 여부를 한 번 집계한다.

Q6 튜닝 후



구분	구매고객	30일 내 재구매고객	재구매율	Execution Time
튜닝 전	_____	_____	_____%	_____ ms
튜닝 후	_____	_____	_____%	_____ ms

결과 해석: _____

9. Q7 - 재주문 임계치 미만 상품

요청사항

qty_on_hand < reorder_point 인 상품과 부족수량을 제공한다.

Q7 튜닝 전




```
-- =====
-- Q8. 평점 4.5 이상, 리뷰 50개 이상 효자상품
-- 문제점: 상품마다 AVG와 COUNT를 SELECT와 WHERE에서 반복 계산한다.
-- =====

-- [① 실행계획 확인용] 제품별 리뷰 반복 집계 횟수 확인
EXPLAIN (ANALYZE, BUFFERS, SUMMARY)
SELECT p.product_id,
       p.product_name,
       ROUND((SELECT AVG(r.rating) FROM ecom.reviews r
              WHERE r.product_id = p.product_id), 2) AS avg_rating, -- 제품별 평균평점
       (SELECT COUNT(r.review_id) FROM ecom.reviews r
        WHERE r.product_id = p.product_id) AS review_count -- 제품별 리뷰 수
FROM ecom.products p
WHERE (SELECT AVG(r.rating) FROM ecom.reviews r
      WHERE r.product_id = p.product_id) >= 4.5
      AND (SELECT COUNT(r.review_id) FROM ecom.reviews r
          WHERE r.product_id = p.product_id) >= 50
ORDER BY avg_rating DESC, review_count DESC, p.product_id;
```

SELECT p.product_id, p.product_name, ROUND((SELECT AVG(r.rating) FROM ecom.reviews r WHERE r.product_id = p.product_id), 2) AS avg_rating, (SELECT COUNT(r.review_id) FROM ecom.reviews r WHERE r.product_id = p.product_id) AS review_count FROM ecom.products p WHERE (SELECT AVG(r.rating) FROM ecom.reviews r WHERE r.product_id = p.product_id) >= 4.5 AND (SELECT COUNT(r.review_id) FROM ecom.reviews r WHERE r.product_id = p.product_id) >= 50 ORDER BY avg_rating DESC, review_count DESC, p.product_id;

id	product_id	product_name	avg_rating	review_count
1	57	Product 491	4.85	160
2	229	Product 464	4.83	160
3	163	Product 253	4.81	160
4	263	Product 285	4.79	160
5	426	Product 258	4.79	160
6	243	Product 395	4.78	160
7	268	Product 25	4.77	160
8	297	Product 75	4.76	160
9	481	Product 189	4.76	160
10	356	Product 556	4.75	161
11	178	Product 113	4.74	160
12	114	Product 262	4.69	160

EXPLAIN (ANALYZE, BUFFERS, SUMMARY) SELECT p.product_id, p.product_name, ROUND((SELECT AVG(r.rating) FROM ecom.reviews r WHERE r.product_id = p.product_id), 2) AS avg_rating, (SELECT COUNT(r.review_id) FROM ecom.reviews r WHERE r.product_id = p.product_id) AS review_count FROM ecom.products p WHERE (SELECT AVG(r.rating) FROM ecom.reviews r WHERE r.product_id = p.product_id) >= 4.5 AND (SELECT COUNT(r.review_id) FROM ecom.reviews r WHERE r.product_id = p.product_id) >= 50 ORDER BY avg_rating DESC, review_count DESC, p.product_id;

id	product_id	product_name	avg_rating	review_count
1	57	Product 491	4.85	160
2	229	Product 464	4.83	160
3	163	Product 253	4.81	160
4	263	Product 285	4.79	160
5	426	Product 258	4.79	160
6	243	Product 395	4.78	160
7	268	Product 25	4.77	160
8	297	Product 75	4.76	160
9	481	Product 189	4.76	160
10	356	Product 556	4.75	161
11	178	Product 113	4.74	160
12	114	Product 262	4.69	160

느린 이유와 개선 방법

튜닝 전에는 제품별 평균과 건수를 SELECT와 WHERE에서 반복 계산한다. 튜닝 후에는 리뷰를 한 번 읽고 **GROUP BY** 와 **HAVING** 으로 조건을 적용한다.

Q8 튜닝 후

Q8. 리뷰 데이터를 한 번만 읽고 GROUP BY/HAVING 적용
-- 튜닝: 제품별 AVG와 COUNT를 한 번 계산하고 HAVING에서 두 조건을 함께 적용한다.
-- [① 실행계획 확인용] reviews를 한 번만 집계하는지 확인

EXPLAIN (ANALYZE, BUFFERS, SUMMARY)

SELECT p.product_id, p.product_name, ROUND(AVG(r.rating), 2) AS avg_rating, COUNT(r.review_id) AS review_count FROM ecom.reviews r JOIN ecom.products p ON p.product_id = r.product_id GROUP BY p.product_id, p.product_name HAVING AVG(r.rating) >= 4.5 AND COUNT(r.review_id) >= 50 ORDER BY avg_rating DESC, review_count DESC, p.product_id;

Results 1 X

EXPLAIN (ANALYZE, BUFFERS, SUMMARY) SELECT p.product_id, p.product_name, ROUND(AVG(r.rating), 2) AS avg_rating, COUNT(r.review_id) AS review_count FROM ecom.reviews r JOIN ecom.products p ON p.product_id = r.product_id GROUP BY p.product_id, p.product_name HAVING AVG(r.rating) >= 4.5 AND COUNT(r.review_id) >= 50 ORDER BY avg_rating DESC, review_count DESC, p.product_id;

id	product_id	product_name	avg_rating	review_count
1	57	Product 491	4.85	160
2	229	Product 464	4.83	160
3	163	Product 253	4.81	160
4	263	Product 285	4.79	160
5	426	Product 258	4.79	160
6	243	Product 395	4.78	160
7	268	Product 25	4.77	160
8	297	Product 75	4.76	160
9	481	Product 189	4.76	160
10	356	Product 556	4.75	161
11	178	Product 113	4.74	160
12	114	Product 262	4.69	160

SELECT p.product_id, p.product_name, ROUND(AVG(r.rating), 2) AS avg_rating, COUNT(r.review_id) AS review_count FROM ecom.reviews r JOIN ecom.products p ON p.product_id = r.product_id GROUP BY p.product_id, p.product_name HAVING AVG(r.rating) >= 4.5 AND COUNT(r.review_id) >= 50 ORDER BY avg_rating DESC, review_count DESC, p.product_id;

products 1 X

SELECT p.product_id, p.product_name, ROUND(AVG(r.rating), 2) AS avg_rating, COUNT(r.review_id) AS review_count FROM ecom.reviews r JOIN ecom.products p ON p.product_id = r.product_id GROUP BY p.product_id, p.product_name HAVING AVG(r.rating) >= 4.5 AND COUNT(r.review_id) >= 50 ORDER BY avg_rating DESC, review_count DESC, p.product_id;

id	product_id	product_name	avg_rating	review_count
1	57	Product 491	4.85	160
2	229	Product 464	4.83	160
3	163	Product 253	4.81	160
4	263	Product 285	4.79	160
5	426	Product 258	4.79	160
6	243	Product 395	4.78	160
7	268	Product 25	4.77	160
8	297	Product 75	4.76	160
9	481	Product 189	4.76	160
10	356	Product 556	4.75	161
11	178	Product 113	4.74	160
12	114	Product 262	4.69	160

구분	Reviews 접근 방식	Buffers	Execution Time	조건 충족 상품 수
튜닝 전	_____	_____	_____ ms	_____
튜닝 후	_____	_____	_____ ms	_____

결과 해석: _____

11. Q9 - 쿠폰 사용 여부별 AOV

요청사항

쿠폰 사용 주문과 미사용 주문의 주문당 평균 금액을 비교한다.

Q9 튜닝 전

```
-- Q9. 쿠폰 사용 여부별 평균 주문금액
-- 문제점: 상세 행 조인 후 COUNT(DISTINCT)로 주문 수를 복원한다.
-- 의미: AOV는 전체 매출을 중복 없는 주문 수로 나눈 주문당 평균금액이다.

-- [① 실행계획 확인용] 상세행 조인과 DISTINCT 집계 비용 확인
EXPLAIN (ANALYZE, BUFFERS, SUMMARY)
SELECT CASE WHEN o.coupon_code IS NULL THEN '미사용' ELSE '사용' END AS coupon_group, -- 쿠폰 여부 분류
COUNT(DISTINCT o.order_id) AS order_count, -- 중복을 제거한 주문 수
ROUND(SUM(oi.line_total) / NULLIF(COUNT(DISTINCT o.order_id), 0), 2) AS aov -- 주문당 평균금액
FROM ecom.orders o
JOIN ecom.order_items oi ON oi.order_id = o.order_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered')
GROUP BY 1
ORDER BY 1;
```

```
= SELECT CASE WHEN o.coupon_code IS NULL THEN '미사용' ELSE '사용' END AS coupon_group, -- [①] 실제 쿠폰 여부
COUNT(DISTINCT o.order_id) AS order_count,
ROUND(SUM(oi.line_total) / NULLIF(COUNT(DISTINCT o.order_id), 0), 2) AS aov
FROM ecom.orders o
JOIN ecom.order_items oi ON oi.order_id = o.order_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered')
GROUP BY 1
ORDER BY 1;
```

coupon_group	order_count	aov
미사용	2,307	619.55
사용	761	1,694.79

Results 1 X
EXPLAIN (ANALYZE, BUFFERS, SUMMARY) SELECT CASE WHEN o.coupon_code IS NULL THEN '미사용' ELSE '사용' END AS coupon_group, COUNT(DISTINCT o.order_id) AS order_count, ROUND(SUM(oi.line_total) / NULLIF(COUNT(DISTINCT o.order_id), 0), 2) AS aov FROM ecom.orders o JOIN ecom.order_items oi ON oi.order_id = o.order_id WHERE o.order_status IN ('paid', 'shipped', 'delivered') GROUP BY 1 ORDER BY 1;
Grid
1 Az QUERY PLAN
2 GroupAggregate (cost=898.29..980.71 rows=1 width=72) (actual time=17.358..17.942 rows=2 loops=1)
3 Group Key: (CASE WHEN (o.coupon_code IS NULL) THEN '미사용'::text ELSE '사용'::text END)
4 Buffers: shared hit=143
5 -> Sort (cost=898.29..918.89 rows=8240 width=46) (actual time=16.192..16.553 rows=8230 loops=1)
6 Sort Key: (CASE WHEN (o.coupon_code IS NULL) THEN '미사용'::text ELSE '사용'::text END), o.order_id
7 Sort Method: quicksort Memory: 750kB
8 Buffers: shared hit=143
9 -> Hash Join (cost=131.43..362.34 rows=8240 width=46) (actual time=1.732..9.532 rows=8230 loops=1)
10 Hash Cond: (oi.order_id = o.order_id)
11 Buffers: shared hit=140
12 -> Seq Scan on order_items oi (cost=0.00..203.67 rows=10367 width=14) (actual time=0.006..5.943 rows=10367 loops=1)
13 Buffers: shared hit=100
14 -> Hash (cost=93.08..93.08 rows=3068 width=15) (actual time=1.702..1.703 rows=3068 loops=1)
15 Buckets: 4096 Batches: 1 Memory Usage: 158kB
16 Buffers: shared hit=40
17 -> Seq Scan on orders o (cost=0.00..93.08 rows=3068 width=15) (actual time=0.012..1.193 rows=3068 loops=1)
18 Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
19 Rows Removed by Filter: 792
20 Buffers: shared hit=40
21 Planning:
22 Buffers: shared hit=150 dirtied=1
23 Planning Time: 4.284 ms
24 Execution Time: 17.993 ms

느린 이유와 개선 방법

상세행 조인 후에는 주문 수 복원을 위해 **COUNT(DISTINCT)** 가 필요하다. 튜닝 후에는 주문별 금액을 먼저 계산한 뒤 쿠폰 사용 여부로 그룹핑해 **AVG(order_total)** 을 계산한다.

Q9 튜닝 후

```
-- Q9. 주문별 매출 선집계로 DISTINCT 제거
-- 튜닝: 주문별 금액을 먼저 한 행으로 만든 뒤 그룹별 AVG(order_total)을 계산한다.
-- [① 실행계획 확인용] 주문별 선집계 후 AVG 계산 계획 확인
EXPLAIN (ANALYZE, BUFFERS, SUMMARY)
WITH order_totals AS (
SELECT o.order_id,
(o.coupon_code IS NOT NULL) AS used_coupon, -- 쿠폰 사용 여부를 TRUE/FALSE로 변환
SUM(oi.line_total) AS order_total -- 주문별 총금액
FROM ecom.orders o
JOIN ecom.order_items oi ON oi.order_id = o.order_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered')
GROUP BY o.order_id, o.coupon_code
)
SELECT CASE WHEN used_coupon THEN '사용' ELSE '미사용' END AS coupon_group,
COUNT(order_id) AS order_count,
ROUND(AVG(order_total), 2) AS aov -- 주문별 금액의 평균
FROM order_totals
GROUP BY used_coupon
ORDER BY coupon_group;
```

Results 1 X
EXPLAIN (ANALYZE, BUFFERS, SUMMARY) WITH order_totals AS (SELECT o.order_id, (o.coupon_code IS NOT NULL) AS used_coupon, SUM(oi.line_total) AS order_total FROM ecom.orders o JOIN ecom.order_items oi ON oi.order_id = o.order_id WHERE o.order_status IN ('paid', 'shipped', 'delivered') GROUP BY o.order_id, o.coupon_code) SELECT CASE WHEN used_coupon THEN '사용' ELSE '미사용' END AS coupon_group, COUNT(order_id) AS order_count, ROUND(AVG(order_total), 2) AS aov FROM order_totals GROUP BY used_coupon ORDER BY coupon_group;
Grid
1 Az QUERY PLAN
2 Sort (cost=495.62..495.63 rows=2 width=73) (actual time=9.883..9.885 rows=2 loops=1)
3 Sort Key: (CASE WHEN ((o.coupon_code IS NOT NULL)) THEN '사용'::text ELSE '미사용'::text END)
4 Sort Method: quicksort Memory: 25kB
5 Buffers: shared hit=140
6 -> HashAggregate (cost=495.58..495.61 rows=2 width=73) (actual time=9.788..9.791 rows=2 loops=1)
7 Group Key: (o.coupon_code IS NOT NULL)
8 Batches: 1 Memory Usage: 24kB
9 Buffers: shared hit=140
10 -> HashAggregate (cost=403.54..441.89 rows=3068 width=48) (actual time=8.547..9.389 rows=3068 loops=1)
11 Group Key: o.order_id
12 Batches: 1 Memory Usage: 1393kB
13 Buffers: shared hit=140
14 -> Hash Join (cost=131.43..362.34 rows=8240 width=21) (actual time=1.424..5.397 rows=8230 loops=1)
15 Hash Cond: (oi.order_id = o.order_id)
16 Buffers: shared hit=140
17 -> Seq Scan on order_items oi (cost=0.00..203.67 rows=10367 width=14) (actual time=0.014..1.794 rows=10367 loops=1)
18 Buffers: shared hit=100
19 -> Hash (cost=93.08..93.08 rows=3068 width=15) (actual time=1.349..1.350 rows=3068 loops=1)
20 Buckets: 4096 Batches: 1 Memory Usage: 158kB
21 Buffers: shared hit=40
22 -> Seq Scan on orders o (cost=0.00..93.08 rows=3068 width=15) (actual time=0.008..0.945 rows=3068 loops=1)
23 Filter: (order_status = ANY ('{paid,shipped,delivered}'::text[]))
24 Rows Removed by Filter: 792
25 Buffers: shared hit=40
26 Planning:
27 Buffers: shared hit=22 dirtied=1
28 Planning Time: 0.613 ms
29 Execution Time: 10.057 ms

```
WITH order_totals AS (
SELECT o.order_id, (o.coupon_code IS NOT NULL) AS used_coupon,
SUM(oi.line_total) AS order_total
FROM ecom.orders o
JOIN ecom.order_items oi ON oi.order_id = o.order_id
WHERE o.order_status IN ('paid', 'shipped', 'delivered')
GROUP BY o.order_id, o.coupon_code
)
SELECT CASE WHEN used_coupon THEN '사용' ELSE '미사용' END AS coupon_group,
COUNT(order_id) AS order_count, ROUND(AVG(order_total), 2) AS aov
FROM order_totals
GROUP BY used_coupon
ORDER BY coupon_group;
```

Results 1 X
WITH order_totals AS (SELECT o.order_id, (o.coupon_code IS NOT NULL) AS used_coupon, SUM(oi.line_total) AS order_total FROM ecom.orders o JOIN ecom.order_items oi ON oi.order_id = o.order_id WHERE o.order_status IN ('paid', 'shipped', 'delivered') GROUP BY o.order_id, o.coupon_code) SELECT CASE WHEN used_coupon THEN '사용' ELSE '미사용' END AS coupon_group, COUNT(order_id) AS order_count, ROUND(AVG(order_total), 2) AS aov FROM order_totals GROUP BY used_coupon ORDER BY coupon_group;
Grid
1 Az coupon_group
2 123 order_count
123 aov
1 미사용
2,307
619.55
2 사용
761
1,694.79

그룹	주문 수	AOV	해석
쿠폰 미사용	_____	_____	_____
쿠폰 사용	_____	_____	_____

구분	Buffers	Execution Time
튜닝 전	_____	_____ ms
튜닝 후	_____	_____ ms

쿠폰 사용과 높은 AOV의 상관관계만 확인할 수 있으며, 쿠폰의 인과효과를 의미하지는 않는다.

12. Q10 - 누적매출 상위 1% 고객의 최근 60일 매출

요청사항

전체 고객 3,000명을 누적매출로 정렬해 상위 1%인 정확히 30명을 먼저 선정하고, 선정된 고객의 최근 60일 매출을 제공한다.

Q10 튜닝 전

=====	
-- Q10. 누적매출 상위 1% 고객의 최근 60일 매출	
-- 문제점: 3,000명 각각의 누적매출을 상관 서브쿼리로 계산하고,	
-- 선정된 상위 30명의 최근 60일 매출도 다시 상관 조회한다.	
-- 기준: 전체 고객 3,000명의 누적매출로 정확히 30명을 먼저 선정한다.	
-- 주의: 누적매출로 30명을 먼저 선정한 다음, 그 고객들의 최근 60일 매출을 표시한다.	
=====	
-- [① 실행계획 확인용] 고객별 반복 집계와 최근 60일 SubPlan 비용 확인	
EXPLAIN (ANALYZE, BUFFERS, SUMMARY)	-- 고객별 반복 집계 비용 측정
WITH lifetime_sales AS (	-- 고객별 누적매출 CTE
SELECT c.customer_id,	
c.full_name,	
(	
SELECT COALESCE(SUM(oi.line_total), 0) -- 고객 한 명의 누적매출 계산	
FROM ecom.orders o	
JOIN ecom.order_items oi ON oi.order_id = o.order_id	
WHERE o.customer_id = c.customer_id	
AND o.order_status IN ('paid', 'shipped', 'delivered')	
) AS lifetime_revenue	
FROM ecom.customers c	
), ranked AS (	-- 누적매출 순위 계산 CTE
SELECT customer_id,	
full_name,	
lifetime_revenue,	
ROW_NUMBER() OVER (	-- 중복 없는 순번을 1부터 부여
ORDER BY lifetime_revenue DESC, customer_id	
) AS lifetime_rank,	
COUNT(customer_id) OVER () AS customer_count -- 전체 고객 수를 각 행에 표시	
FROM lifetime_sales	
)	
SELECT r.customer_id,	
r.full_name,	
r.lifetime_rank,	
ROUND(r.lifetime_revenue, 2) AS lifetime_revenue,	
ROUND((	
SELECT COALESCE(SUM(oi.line_total), 0)	
FROM ecom.orders o	
JOIN ecom.order_items oi ON oi.order_id = o.order_id	
WHERE o.customer_id = r.customer_id	
AND o.order_status IN ('paid', 'shipped', 'delivered')	
AND o.order_ts >= now() - INTERVAL '60 days' -- 선정 고객의 최근 60일 매출	
), 2) AS revenue_60d	
FROM ranked r	
WHERE r.lifetime_rank <= CEIL(r.customer_count * 0.01) -- 전체 고객의 상위 1%, 정확히 30명	
ORDER BY r.lifetime_rank;	

=====	
-- Q10. 누적매출 상위 1% 고객의 최근 60일 매출	
-- 문제점: 3,000명 각각의 누적매출을 상관 서브쿼리로 계산하고,	
-- 선정된 상위 30명의 최근 60일 매출도 다시 상관 조회한다.	
-- 기준: 전체 고객 3,000명의 누적매출로 정확히 30명을 먼저 선정한다.	
-- 주의: 누적매출로 30명을 먼저 선정한 다음, 그 고객들의 최근 60일 매출을 표시한다.	
=====	
-- [① 실행계획 확인용] 고객별 반복 집계와 최근 60일 SubPlan 비용 확인	
EXPLAIN (ANALYZE, BUFFERS, SUMMARY)	-- 고객별 반복 집계 비용 측정
WITH lifetime_sales AS (	-- 고객별 누적매출 CTE
SELECT c.customer_id,	
c.full_name,	
(	
SELECT COALESCE(SUM(oi.line_total), 0) -- 고객 한 명의 누적매출 계산	
FROM ecom.orders o	
JOIN ecom.order_items oi ON oi.order_id = o.order_id	
WHERE o.customer_id = c.customer_id	
AND o.order_status IN ('paid', 'shipped', 'delivered')	
) AS lifetime_revenue	
FROM ecom.customers c	
), ranked AS (	-- 누적매출 순위 계산 CTE
SELECT customer_id,	
full_name,	
lifetime_revenue,	
ROW_NUMBER() OVER (	-- 중복 없는 순번을 1부터 부여
ORDER BY lifetime_revenue DESC, customer_id	
) AS lifetime_rank,	
COUNT(customer_id) OVER () AS customer_count -- 전체 고객 수를 각 행에 표시	
FROM lifetime_sales	
)	
SELECT r.customer_id,	
r.full_name,	
r.lifetime_rank,	
ROUND(r.lifetime_revenue, 2) AS lifetime_revenue,	
ROUND((	
SELECT COALESCE(SUM(oi.line_total), 0)	
FROM ecom.orders o	
JOIN ecom.order_items oi ON oi.order_id = o.order_id	
WHERE o.customer_id = r.customer_id	
AND o.order_status IN ('paid', 'shipped', 'delivered')	
AND o.order_ts >= now() - INTERVAL '60 days' -- 선정 고객의 최근 60일 매출	
), 2) AS revenue_60d	
FROM ranked r	
WHERE r.lifetime_rank <= CEIL(r.customer_count * 0.01) -- 전체 고객의 상위 1%, 정확히 30명	
ORDER BY r.lifetime_rank;	

=====	
-- Q10. 누적매출 상위 1% 고객의 최근 60일 매출	
-- 문제점: 3,000명 각각의 누적매출을 상관 서브쿼리로 계산하고,	
-- 선정된 상위 30명의 최근 60일 매출도 다시 상관 조회한다.	
-- 기준: 전체 고객 3,000명의 누적매출로 정확히 30명을 먼저 선정한다.	
-- 주의: 누적매출로 30명을 먼저 선정한 다음, 그 고객들의 최근 60일 매출을 표시한다.	
=====	
-- [① 실행계획 확인용] 고객별 반복 집계와 최근 60일 SubPlan 비용 확인	
EXPLAIN (ANALYZE, BUFFERS, SUMMARY)	-- 고객별 반복 집계 비용 측정
WITH lifetime_sales AS (	-- 고객별 누적매출 CTE
SELECT c.customer_id,	
c.full_name,	
(	
SELECT COALESCE(SUM(oi.line_total), 0) -- 고객 한 명의 누적매출 계산	
FROM ecom.orders o	
JOIN ecom.order_items oi ON oi.order_id = o.order_id	
WHERE o.customer_id = c.customer_id	
AND o.order_status IN ('paid', 'shipped', 'delivered')	
) AS lifetime_revenue	
FROM ecom.customers c	
), ranked AS (	-- 누적매출 순위 계산 CTE
SELECT customer_id,	
full_name,	
lifetime_revenue,	
ROW_NUMBER() OVER (	-- 중복 없는 순번을 1부터 부여
ORDER BY lifetime_revenue DESC, customer_id	
) AS lifetime_rank,	
COUNT(customer_id) OVER () AS customer_count -- 전체 고객 수를 각 행에 표시	
FROM lifetime_sales	
)	
SELECT r.customer_id,	
r.full_name,	
r.lifetime_rank,	
ROUND(r.lifetime_revenue, 2) AS lifetime_revenue,	
ROUND((	
SELECT COALESCE(SUM(oi.line_total), 0)	
FROM ecom.orders o	
JOIN ecom.order_items oi ON oi.order_id = o.order_id	
WHERE o.customer_id = r.customer_id	
AND o.order_status IN ('paid', 'shipped', 'delivered')	
AND o.order_ts >= now() - INTERVAL '60 days' -- 선정 고객의 최근 60일 매출	
), 2) AS revenue_60d	
FROM ranked r	
WHERE r.lifetime_rank <= CEIL(r.customer_count * 0.01) -- 전체 고객의 상위 1%, 정확히 30명	
ORDER BY r.lifetime_rank;	

=====	
-- Q10. 누적매출 상위 1% 고객의 최근 60일 매출	
-- 문제점: 3,000명 각각의 누적매출을 상관 서브쿼리로 계산하고,	
-- 선정된 상위 30명의 최근 60일 매출도 다시 상관 조회한다.	
-- 기준: 전체 고객 3,000명의 누적매출로 정확히 30명을 먼저 선정한다.	
-- 주의: 누적매출로 30명을 먼저 선정한 다음, 그 고객들의 최근 60일 매출을 표시한다.	
=====	
-- [① 실행계획 확인용] 고객별 반복 집계와 최근 60일 SubPlan 비용 확인	
EXPLAIN (ANALYZE, BUFFERS, SUMMARY)	-- 고객별 반복 집계 비용 측정
WITH lifetime_sales AS (	-- 고객별 누적매출 CTE
SELECT c.customer_id,	
c.full_name,	
(	
SELECT COALESCE(SUM(oi.line_total), 0) -- 고객 한 명의 누적매출 계산	
FROM ecom.orders o	
JOIN ecom.order_items oi ON oi.order_id = o.order_id	
WHERE o.customer_id = c.customer_id	
AND o.order_status IN ('paid', 'shipped', 'delivered')	
) AS lifetime_revenue	
FROM ecom.customers c	
), ranked AS (	-- 누적매출 순위 계산 CTE
SELECT customer_id,	
full_name,	
lifetime_revenue,	
ROW_NUMBER() OVER (	-- 중복 없는 순번을 1부터 부여
ORDER BY lifetime_revenue DESC, customer_id	
) AS lifetime_rank,	
COUNT(customer_id) OVER () AS customer_count -- 전체 고객 수를 각 행에 표시	
FROM lifetime_sales	
)	
SELECT r.customer_id,	
r.full_name,	
r.lifetime_rank,	
ROUND(r.lifetime_revenue, 2) AS lifetime_revenue,	
ROUND((	
SELECT COALESCE(SUM(oi.line_total), 0)	
FROM ecom.orders o	
JOIN ecom.order_items oi ON oi.order_id = o.order_id	
WHERE o.customer_id = r.customer_id	
AND o.order_status IN ('paid', 'shipped', 'delivered')	
AND o.order_ts >= now() - INTERVAL '60 days' -- 선정 고객의 최근 60일 매출	
), 2) AS revenue_60d	
FROM ranked r	
WHERE r.lifetime_rank <= CEIL(r.customer_count * 0.01) -- 전체 고객의 상위 1%, 정확히 30명	
ORDER BY r.lifetime_rank;	

=====	
-- Q10. 누적매출 상위 1% 고객의 최근 60일 매출	
-- 문제점: 3,000명 각각의 누적매출을 상관 서브쿼리로 계산하고,	
-- 선정된 상위 30명의 최근 60일 매출도 다시 상관 조회한다.	
-- 기준: 전체 고객 3,000명의 누적매출로 정확히 30명을 먼저 선정한다.	
-- 주의: 누적매출로 30명을 먼저 선정한 다음, 그 고객들의 최근 60일 매출을 표시한다.	
=====	
-- [① 실행계획 확인용] 고객별 반복 집계와 최근 60일 SubPlan 비용 확인	
EXPLAIN (ANALYZE, BUFFERS, SUMMARY)	-- 고객별 반복 집계 비용 측정
WITH lifetime_sales AS (	-- 고객별 누적매출 CTE
SELECT c.customer_id,	
c.full_name,	
(	
SELECT COALESCE(SUM(oi.line_total), 0) -- 고객 한 명의 누적매출 계산	
FROM ecom.orders o	
JOIN ecom.order_items oi ON oi.order_id = o.order_id	
WHERE o.customer_id = c.customer_id	
AND o.order_status IN ('paid', 'shipped', 'delivered')	
) AS lifetime_revenue	
FROM ecom.customers c	
), ranked AS (	-- 누적매출 순위 계산 CTE
SELECT customer_id,	
full_name,	
lifetime_revenue,	
ROW_NUMBER() OVER (	-- 중복 없는 순번을 1부터 부여
ORDER BY lifetime_revenue DESC, customer_id	
) AS lifetime_rank,	
COUNT(customer_id) OVER () AS customer_count -- 전체 고객 수를 각 행에 표시	
FROM lifetime_sales	
)	
SELECT r.customer_id,	
r.full_name,	
r.lifetime_rank,	
ROUND(r.lifetime_revenue, 2) AS lifetime_revenue,	
ROUND((	
SELECT COALESCE(SUM(oi.line_total), 0)	
FROM ecom.orders o	
JOIN ecom.order_items oi ON oi.order_id = o.order_id	
WHERE o.customer_id = r.customer_id	
AND o.order_status IN ('paid', 'shipped', 'delivered')	
AND o.order_ts >= now() - INTERVAL '60 days' -- 선정 고객의 최근 60일 매출	
), 2) AS revenue_60d	
FROM ranked r	
WHERE r.lifetime_rank <= CEIL(r.customer_count * 0.01) -- 전체 고객의 상위 1%, 정확히 30명	
ORDER BY r.lifetime_rank;	

느린 이유와 개선 방법

튜닝 전에는 고객 3,000명의 누적매출과 상위 고객의 최근 매출을 상관 서브쿼리로 반복 계산한다. 튜닝 후에는 전체 고객을 기준으로 누적매출과 최근 60일 매출을 한 번에 집계하고, `ROW_NUMBER()` 와 전체 고객 수를 이용해 30명을 정확히 추출한다.

Q10 튜닝 후

```
-- Q10. 전체 3,000명의 누적매출과 최근 60일 매출을 한 번에 집계한 뒤
-- 누적매출 상위 1%(30명)를 정확히 선정
-- 튜닝: 누적매출과 60일 매출을 한 번의 GROUP BY로 동시에 계산한다.
-- [⓪ 실행계획 확인용] 고객별 SubPlan 없이 한 번에 집계하는지 확인
EXPLAIN (ANALYZE, BUFFERS, SUMMARY)
WITH customer_sales AS (
  SELECT c.customer_id,
         c.full_name,
         COALESCE(SUM(oi.line_total), 0) AS lifetime_revenue, -- 고객 누적매출
         COALESCE(SUM(oi.line_total) FILTER (
           WHERE o.order_ts >= now() - INTERVAL '60 days'
         ), 0) AS revenue_60d
    FROM ecom.customers c
   LEFT JOIN ecom.orders o
     ON o.customer_id = c.customer_id
    AND o.order_status IN ('paid', 'shipped', 'delivered')
   LEFT JOIN ecom.order_items oi ON oi.order_id = o.order_id
  GROUP BY c.customer_id, c.full_name
), ranked AS (
  SELECT customer_id, full_name, lifetime_revenue, revenue_60d,
         ROW_NUMBER() OVER (
           ORDER BY lifetime_revenue DESC, customer_id
         ) AS lifetime_rank,
         COUNT(customer_id) OVER () AS customer_count -- 전체 고객 수를 각 행에 표시
    FROM customer_sales
)
SELECT customer_id, full_name, lifetime_rank,
       ROUND(lifetime_revenue, 2) AS lifetime_revenue,
       ROUND(revenue_60d, 2) AS revenue_60d
  FROM ranked
 WHERE lifetime_rank <= CEIL(customer_count * 0.01) -- 전체 고객 중 상위 1%, 정확히 30명
 ORDER BY lifetime_rank;
```

```
-- [⓪] 실제 상위 30명 결과 조회
WITH customer_sales AS (
  SELECT c.customer_id, c.full_name,
         COALESCE(SUM(oi.line_total), 0) AS lifetime_revenue,
         COALESCE(SUM(oi.line_total) FILTER (
           WHERE o.order_ts >= now() - INTERVAL '60 days'
         ), 0) AS revenue_60d
    FROM ecom.customers c
   LEFT JOIN ecom.orders o
     ON o.customer_id = c.customer_id
    AND o.order_status IN ('paid', 'shipped', 'delivered')
   LEFT JOIN ecom.order_items oi ON oi.order_id = o.order_id
  GROUP BY c.customer_id, c.full_name
), ranked AS (
  SELECT customer_id, full_name, lifetime_revenue, revenue_60d,
         ROW_NUMBER() OVER (
           ORDER BY lifetime_revenue DESC, customer_id
         ) AS lifetime_rank,
         COUNT(customer_id) OVER () AS customer_count
    FROM customer_sales
)
SELECT customer_id, full_name, lifetime_rank,
       ROUND(lifetime_revenue, 2) AS lifetime_revenue,
       ROUND(revenue_60d, 2) AS revenue_60d
  FROM ranked
 WHERE lifetime_rank <= CEIL(customer_count * 0.01)
 ORDER BY lifetime_rank;
```

customers 1 X					
WITH customer_sales AS (SELECT c.customer_id, c.full_name, COALESCE(SUM(oi.line_total), 0) AS lifetime_revenue, COALESCE(SUM(oi.line_total) FILTER (WHERE o.order_ts >= now() - INTERVAL '60 days'), 0) AS revenue_60d FROM ecom.customers c LEFT JOIN ecom.orders o ON o.customer_id = c.customer_id AND o.order_status IN ('paid', 'shipped', 'delivered') LEFT JOIN ecom.order_items oi ON oi.order_id = o.order_id GROUP BY c.customer_id, c.full_name), ranked AS (SELECT customer_id, full_name, lifetime_revenue, revenue_60d, ROW_NUMBER() OVER (ORDER BY lifetime_revenue DESC, customer_id) AS lifetime_rank, COUNT(customer_id) OVER () AS customer_count FROM customer_sales) SELECT customer_id, full_name, lifetime_rank, ROUND(lifetime_revenue, 2) AS lifetime_revenue, ROUND(revenue_60d, 2) AS revenue_60d FROM ranked WHERE lifetime_rank <= CEIL(customer_count * 0.01) ORDER BY lifetime_rank;					
	customer_id	full_name	lifetime_rank	lifetime_revenue	revenue_60d
1	8	Customer 8	1	35,154.22	32,794.66
2	16	Customer 16	2	31,003.24	31,003.24
3	18	Customer 18	3	30,283.04	29,772.64
4	14	Customer 14	4	29,322.59	29,322.59
5	6	Customer 6	5	28,703.23	28,703.23
6	13	Customer 13	6	28,633.41	28,633.41
7	2	Customer 2	7	28,623.31	28,623.31
8	1	Customer 1	8	28,546.26	28,546.26
9	20	Customer 20	9	28,531.13	28,531.13
10	9	Customer 9	10	28,488.91	28,488.91
11	17	Customer 17	11	27,317.53	26,943.11
12	24	Customer 24	12	27,075.74	27,075.74
13	23	Customer 23	13	25,717.57	25,717.57
14	30	Customer 30	14	24,815.39	24,815.39
15	4	Customer 4	15	24,726.24	24,101.82
16	12	Customer 12	16	22,419.91	21,470.83
17	19	Customer 19	17	22,412.27	22,412.27
18	22	Customer 22	18	22,206.5	21,128.93
19	7	Customer 7	19	22,034.39	22,034.39
20	29	Customer 29	20	21,696.95	21,696.95
21	11	Customer 11	21	21,635.89	20,199.13
22	5	Customer 5	22	21,557.53	21,557.53
23	15	Customer 15	23	21,466.9	21,466.9
24	21	Customer 21	24	21,430.97	21,430.97
25	28	Customer 28	25	20,490.17	19,053.41
26	26	Customer 26	26	18,974.16	18,974.16
27	3	Customer 3	27	18,431.44	18,220.72
28	10	Customer 10	28	17,990.28	17,990.28
29	27	Customer 27	29	17,135.65	17,135.65
30	25	Customer 25	30	15,993.86	15,993.86

Results 1 X

EXPLAIN (ANALYZE, BUFFERS, SUMMARY) WITH customer_sales AS (SELECT c.customer_id, c.full_name, COALESCE(SUM(oi.line_total), 0) AS lifetime_revenue, COALESCE(SUM(oi.line_total) FILTER (WHERE o.order_ts >= now() - INTERVAL '60 days'), 0) AS revenue_60d FROM ecom.customers c LEFT JOIN ecom.orders o ON o.customer_id = c.customer_id AND o.order_status IN ('paid', 'shipped', 'delivered') LEFT JOIN ecom.order_items oi ON oi.order_id = o.order_id GROUP BY c.customer_id, c.full_name), ranked AS (SELECT customer_id, full_name, lifetime_revenue, revenue_60d, ROW_NUMBER() OVER (ORDER BY lifetime_revenue DESC, customer_id) AS lifetime_rank, COUNT(customer_id) OVER () AS customer_count FROM customer_sales) SELECT customer_id, full_name, lifetime_rank, ROUND(lifetime_revenue, 2) AS lifetime_revenue, ROUND(revenue_60d, 2) AS revenue_60d FROM ranked WHERE lifetime_rank <= CEIL(customer_count * 0.01) ORDER BY lifetime_rank;

QUERY PLAN

1

Sort (cost=1053.20..1055.70 rows=1000 width=93) (actual time=16.928..16.932 rows=30 loops=1)

2

Sort Key: ranked.lifetime_rank

3

Sort Method: quicksort Memory: 27kB

4

Buffers: shared hit=180

5

-> Subquery Scan on ranked (cost=930.83..1003.37 rows=1000 width=93) (actual time=15.860..16.914 rows=30 loops=1)

6

Filter: (((ranked.lifetime_rank)::numeric <= ceil(((ranked.customer_count)::numeric * 0.01)))

7

Rows Removed by Filter: 2970

8

Buffers: shared hit=180

9

-> WindowAgg (cost=930.83..930.87 rows=3000 width=101) (actual time=15.857..16.256 rows=3000 loops=1)

10

Buffers: shared hit=180

11

-> WindowAgg (cost=833.39..893.37 rows=3000 width=93) (actual time=14.683..15.357 rows=3000 loops=1)

12

Buffers: shared hit=180

13

-> Sort (cost=833.37..840.87 rows=3000 width=85) (actual time=14.679..14.788 rows=3000 loops=1)

14

Sort Key: customer_sales.lifetime_revenue DESC, customer_sales.customer_id

15

Sort Method: quicksort Memory: 246kB

16

Buffers: shared hit=180

17

-> Subquery Scan on customer_sales (cost=615.11..660.11 rows=3000 width=85) (actual time=12.711..13.595 rows=3000 loops=1)

18

Buffers: shared hit=180

19

-> HashAggregate (cost=615.11..660.11 rows=3000 width=85) (actual time=12.710..13.399 rows=3000 loops=1)

20

Group Key: c.customer_id

21

Batches: 1 Memory Usage: 1905kB

22

Buffers: shared hit=180

23

-> Hash Right Join (cost=238.93..491.51 rows=8240 width=35) (actual time=3.716..8.317 rows=9013 loops=1)

24

Hash Cond: (o.customer_id = c.customer_id)

25

Buffers: shared hit=180

26

-> Hash Right Join (cost=131.43..362.34 rows=8240 width=22) (actual time=1.959..4.969 rows=8230 loops=1)

27

Hash Cond: (oi.order_id = o.order_id)

28

Buffers: shared hit=140

29

-> Seq Scan on order_items oi (cost=0.00..203.67 rows=10367 width=14) (actual time=0.021..1.052 rows=10367 loop

30

Buffers: shared hit=100

31

-> Hash (cost=93.08..93.08 rows=3068 width=24) (actual time=1.912..1.912 rows=3068 loops=1)

32

Buckets: 4096 Batches: 1 Memory Usage: 200kB

33

Buffers: shared hit=40

34

-> Seq Scan on orders o (cost=0.00..93.08 rows=3068 width=24) (actual time=0.008..1.308 rows=3068 loops=1)

35

Filter: (order_status = ANY ('{paid,shipped,delivered}'))::text[]]

36

Rows Removed by Filter: 792

37

Buffers: shared hit=40

38

-> Hash (cost=70.00..70.00 rows=3000 width=21) (actual time=1.717..1.718 rows=3000 loops=1)

39

Buckets: 4096 Batches: 1 Memory Usage: 190kB

40

Buffers: shared hit=40

41

-> Seq Scan on customers c (cost=0.00..70.00 rows=3000 width=21) (actual time=0.028..0.929 rows=3000 loops=1)

42

Buffers: shared hit=40

Planning:

Buffers: shared hit=33

Planning Time: 1.384 ms

Execution Time: 17.230 ms

검증 항목	기대값	실제값
전체 고객 수	3,000	_____
상위 1% 고객 수	30	_____
누적매출 1위 고객	-	_____
해당 고객 최근 60일 매출	-	_____

구분	반복 SubPlan	Buffers	Execution Time
튜닝 전	_____	_____	_____ ms
튜닝 후	_____	_____	_____ ms

결과 해석: _____

13. Q11 - 안전 나눗셈 함수 비교

함수 정의 차이

함수	정의 파일	분모가 0일 때
<code>ecom.f_safe_div</code>	스키마 파일	<code>0</code> 반환
<code>ecom.safe_div</code>	시드 파일	<code>NULL</code> 반환

Q11 함수 비교

— Q11. 스키마와 시드에 정의된 두 함수의 정상 분모 결과 비교
— [① 정상 분모] 분모가 0이 아니면 두 함수가 같은 계산 결과를 반환

```
SELECT ecom.f_safe_div(  
    SUM(oi.line_total), COUNT(DISTINCT o.order_id)  
) AS schema_f_safe_div_aov,  
ecom.safe_div(  
    SUM(oi.line_total), COUNT(DISTINCT o.order_id)  
) AS seed_safe_div_aov  
FROM ecom.orders o  
JOIN ecom.order_items oi ON oi.order_id = o.order_id  
WHERE o.order_status IN ('paid', 'shipped', 'delivered');
```

orders(+) 1 X

```
SELECT ecom.f_safe_div( SUM(oi.line_total), COUNT(DISTINCT o.order_id) ) AS schema_f_safe_div_aov, seed_safe_div_aov
```

	123 schema_f_safe_div_aov	123 seed_safe_div_aov
1	886.2541297262	886.2541297262

— 자미경: f_safe_div(0), safe_div는 NULL을 반환한다.
— [0 & 분표] ecom_div의 반환값의 경우 정적 시퀀스
SELECT ecom.f_safe_div(10, 0) AS schema_function_returns_zero, — 0으로 나눈 결과 0
ecom.safe_div(10, 0) AS seed_function_returns_null; — 0으로 나눈 NULL(값 없음)

sults 1 x Statistics 1

LECT ecom.f_safe_div(10, 0) AS schema_function_returns_zero, ecom — Enter a SQL expression to

	123 schema_function_returns_zero	123 seed_function_returns_null
	0	[NULL]

실행 결과

테스트	f_safe_div	safe_div	해석
정상 AOV	_____	_____	분모가 0이 아니면 같은 계산 결과
100 / 0	0	NULL	보고서 요구에 맞는 정책을 선택해야 함

14. 물리 조인 3종 비교

실행 결과 화면

Nested Loop Join

```

SET search_path = ecom, public; -- 객체 이름 앞에 스키마를 생략하면 ecom을 먼저 탐색

-- 공통 업무 정의
-- 최근 30일 동안 paid, shipped, delivered 상태인 주문과 주문상품을 연결하여 총매출을 계산한다.
-- 세 테스트는 업무 로직과 결과가 같고, 물리적인 JOIN 방식만 다르다.

-- =====
-- 1. Nested Loop Join(NLJ)
-- =====
-- 처리 방식: 바깥쪽 입력이 각 행마다 왼쪽 입력에서 일치하는행을 반복해서 찾는다.
-- 유리한 조건: 바깥쪽 결과가 적고, 왼쪽 조인 키에 인덱스가 있을 때 유리하다.
-- 확인할 실행계획: Nested Loop와 안쪽의 Index Scan 또는 Index Only Scan

BEGIN;
SET LOCAL enable_hashjoin = off; -- SET LOCAL이 현재 테스트에만 적용되도록 트랜잭션 시작
SET LOCAL enable_mergejoin = off; -- Hash Join 후보로 비활성화
SET LOCAL enable_nestloop = on; -- Merge Join 후보를 비활성화
-- Nested Loop Join 후보만 활성화

EXPLAIN (ANALYZE, BUFFERS, SETTINGS, SUMMARY) -- 실제 실행시간·버퍼·변경 설정·요약을 함께 출력
SELECT ROUND(SUM(o.i.line_total), 2) AS total_sales_30d -- 주문상품 금액을 합산하고 소수점 둘째 자리까지 표시
FROM ecom.orders o -- 주문 헤더 테이블, 별칭 ts
JOIN ecom.order_items oi -- 주문상품 테이블과 금액이 저장된 상세 테이블, 별칭 oi
ON oi.order_id = o.order_id -- 주문번호가 같은 주문과 주문상품을 연결하는 등등 조건
WHERE o.order_status IN ('paid', 'shipped', 'delivered') -- 실제 매출로 인정하는 주문 상태만 포함
AND o.order_ts >= CURRENT_DATE - 30; -- 오늘을 기준으로 최근 30일 주문만 조회

ROLLBACK; -- 설정 변경을 취소하여 다음 조인 테스트에 영향을 주지 않음

```

Hash Join

2. Hash Join

처리 방식: 작은 입력의 조인 키로 메모리에 해시 테이블을 만들고 큰 입력의 키를 대조한다.

유리한 조건: 정렬되지 않은 대량 데이터의 동등 조인이고 해시 테이블이 메모리에 들어갈 때 유리하다.

확인할 실행계획: Hash Join, Hash, Hash Cond

BEGIN;

SET LOCAL enable_hashjoin = on;

SET LOCAL enable_mergejoin = off;

SET LOCAL enable_nestloop = off;

EXPLAIN (ANALYZE, BUFFERS, SETTINGS, SUMMARY)

SELECT ROUND(SUM(oi.line_total), 2) AS total_sales_30d

FROM ecom.orders o

JOIN ecom.order_items oi

ON oi.order_id = o.order_id

WHERE o.order_status IN ('paid', 'shipped', 'delivered')

AND o.order_ts >= CURRENT_DATE - 30;

ROLLBACK;

Hash Join 테스트용 독립 트랜잭션 시작

Hash Join 후보 활성화

Merge Join 후보 비활성화

Nested Loop Join 후보 비활성화

실제 Hash Join 계획과 실행 통계 출력

세 조인에서 동일해야 하는 최근 30일 총매출

필터 조건을 가진 주문 테이블

주문에 속한 상품별 금액 테이블

Hash Cond로 사용할 주문번호 동등 조건

위소·함몰·생성 상태를 매출에서 제외

최근 30일 범위만 남겨 조인 대상 축소

Hash Join 강제 설정 해제

Results 1

Statistics 1

BEGIN; SET LOCAL enable_hashjoin = on; SET LOCAL enable_mergejoin = off; Enter a SQL expression to filter results (use Ctrl+Space)

AZ QUERY PLAN

1 Aggregate (cost=292.06..292.08 rows=1 width=32) (actual time=11.540..11.541 rows=1 loops=1)

2 Buffers: shared hit=109

3 -> Hash Join (cost=55.14..286.05 rows=2404 width=6) (actual time=0.369..11.260 rows=2538 loops=1)

4 Hash Cond: (oi.order_id = o.order_id)

5 Buffers: shared hit=109

6 -> Seq Scan on order_items oi (cost=0.00..203.67 rows=10367 width=14) (actual time=0.041..9.457 rows=10367 loops=1)

7 Buffers: shared hit=100

8 -> Hash (cost=43.95..43.95 rows=895 width=8) (actual time=0.316..0.316 rows=937 loops=1)

9 Buckets: 1024 Batches: 1 Memory Usage: 45kB

10 Buffers: shared hit=9

11 -> Index Only Scan using idx_orders_sales_ts_order on orders o (cost=0.28..43.95 rows=895 width=8) (actual time=0.090..0.217 rows=937 loops=1)

12 Index Cond: (order_ts >= (CURRENT_DATE - 30))

13 Heap Fetches: 0

14 Buffers: shared hit=9

15 Settings: search_path = 'ecom, public', enable_mergejoin = 'off', enable_nestloop = 'off'

16 Planning Time: 0.294 ms

17 Execution Time: 11.590 ms

Merge Join

3. Merge Join

처리 방식: 양쪽 입력을 조인 키 순서로 정렬한 뒤 앞에서부터 순차적으로 비교한다.

유리한 조건: 입력이 이미 조인 키로 정렬돼 있거나 정렬 비용이 낮은 대량 조인에 유리하다.

확인할 실행계획: Merge Join, Merge Cond, 필요하면 Sort 노드

BEGIN;

SET LOCAL enable_hashjoin = off;

SET LOCAL enable_mergejoin = on;

SET LOCAL enable_nestloop = off;

EXPLAIN (ANALYZE, BUFFERS, SETTINGS, SUMMARY)

SELECT ROUND(SUM(oi.line_total), 2) AS total_sales_30d

FROM ecom.orders o

JOIN ecom.order_items oi

ON oi.order_id = o.order_id

WHERE o.order_status IN ('paid', 'shipped', 'delivered')

AND o.order_ts >= CURRENT_DATE - 30;

ROLLBACK;

Merge Join 테스트용 독립 트랜잭션 시작

Hash Join 후보 비활성화

Merge Join 후보 활성화

Nested Loop Join 후보 비활성화

실제 Merge Join 계획과 실행 통계 출력

동일한 총매출을 계산하여 결과의 일관성 확인

주문 테이블

주문상품 테이블

Merge Cond로 사용할 주문번호 동등 조건

실매출 상태만 포함

최근 30일 주문만 포함

Merge Join 강제 설정 해제

Results 1

Statistics 1

BEGIN; SET LOCAL enable_hashjoin = off; SET LOCAL enable_mergejoin = on; Enter a SQL expression to filter results (use Ctrl+Space)

AZ QUERY PLAN

1 Aggregate (cost=499.06..499.08 rows=1 width=32) (actual time=5.602..5.603 rows=1 loops=1)

2 Buffers: shared hit=131

3 -> Merge Join (cost=88.11..493.05 rows=2404 width=6) (actual time=0.305..5.346 rows=2538 loops=1)

4 Merge Cond: (oi.order_id = o.order_id)

5 Buffers: shared hit=131

6 -> Index Scan using idx_order_items_order on order_items oi (cost=0.29..350.79 rows=10367 width=14) (actual time=0.030..4.060 rows=10367 loops=1)

7 Buffers: shared hit=122

8 -> Sort (cost=87.83..90.07 rows=895 width=8) (actual time=0.269..0.341 rows=937 loops=1)

9 Sort Key: o.order_id

10 Sort Method: quicksort Memory: 25kB

11 Buffers: shared hit=9

12 -> Index Only Scan using idx_orders_sales_ts_order on orders o (cost=0.28..43.95 rows=895 width=8) (actual time=0.035..0.158 rows=937 loops=1)

13 Index Cond: (order_ts >= (CURRENT_DATE - 30))

14 Heap Fetches: 0

15 Buffers: shared hit=9

16 Settings: search_path = 'ecom, public', enable_hashjoin = 'off', enable_nestloop = 'off'

17 Planning:

18 Buffers: shared hit=30

19 Planning Time: 1.964 ms

20 Execution Time: 5.705 ms

구분	처리 방식	일반적으로 유리한 조건	Buffers	Execution Time
Nested Loop Join	외부 행마다 내부 입력 탐색	외부 입력이 작고 내부 인덱스가 있을 때	_____	_____ ms
Hash Join	작은 입력 해시 생성 후 큰 입력 탐색	큰 비정렬 동등 조인	_____	_____ ms
Merge Join	정렬된 두 입력을 순차 병합	양쪽이 이미 정렬된 큰 입력	_____	_____ ms

본인 환경에서의 결론

15. Materialized View 활용

생성 목적

매일 주문과 주문상품을 다시 조인·집계하지 않고, 미리 계산한 일별 주문 수와 GMV를 빠르게 조회한다.

생성 및 결과 화면

MV 생성

종합실습4_제출리포트_작성틀

16

Name	Value
Updated Rows	0
Execute time	0.023s
Start time	Fri Aug 14 17:05:51 KST 2026
Finish time	Fri Aug 14 17:05:51 KST 2026
Query	DROP MATERIALIZED VIEW IF EXISTS ecom.mv_daily_gmv; -- 기존 MV와 그 MV에 속한 인덱스를 함께 제거 -- ===== -- 2. 일별 GMV Materialized View 생성 -- ===== -- WITH NO DATA로 구조와 정의만 먼저 생성하고 데이터는 뒤의 REFRESH에서 적재한다. CREATE MATERIALIZED VIEW ecom.mv_daily_gmv AS -- 일별 집계 결과를 저장할 MV 생성 시작 SELECT date_trunc('day', o.order_ts)::date AS sales_day, -- 주문시각을 일 단위로 자르고 date 형식으로 변환 COUNT(DISTINCT o.order_id) AS order_count, -- 주문상품 조인으로 중복된 주문번호를 제거한 일별 주문 수 ROUND(SUM(oi.line_total), 2) AS gmv -- 주문상품 금액의 일별 합계를 소수점 둘째 자리로 표시 FROM ecom.orders o -- 주문일시-주문상태가 저장된 주문 테이블 JOIN ecom.order_items oi -- 상품별 판매금액이 저장된 주문상품 테이블 ON oi.order_id = o.order_id -- 주문번호가 같은 주문과 주문상품을 연결 WHERE o.order_status IN ('paid', 'shipped', 'delivered') -- 실제 매출로 인정하는 상태만 포함 GROUP BY date_trunc('day', o.order_ts)::date -- 같은 날짜의 주문상품 금액을 한 행으로 집계 WITH NO DATA; -- 생성 시에는 데이터를 넣지 않고 빈 구조만 생성 -- ===== -- 3. 동시 갱신을 위한 UNIQUE 인덱스 생성 -- ===== -- REFRESH MATERIALIZED VIEW CONCURRENTLY를 사용하려면 모든 행을 유일하게 식별할 수 있어야 한다. -- 이 MV는 날짜별로 한 행만 존재하므로 sales_day를 고유키로 사용한다. CREATE UNIQUE INDEX ux_mv_daily_gmv_sales_day -- MV의 날짜 중복을 금지하는 UNIQUE 인덱스 이름 ON ecom.mv_daily_gmv (sales_day); -- 날짜 한 개가 MV의 각 행을 유일하게 식별 -- ===== -- 4. 최초 데이터 적재 및 통계정보 수집 -- ===== -- WITH NO DATA로 만들었으므로 최초 1회는 일반 REFRESH로 데이터를 채워야 한다. -- 빈 MV에는 CONCURRENTLY를 바로 사용할 수 없으므로 먼저 일반 REFRESH를 실행한다. REFRESH MATERIALIZED VIEW ecom.mv_daily_gmv; -- MV 정의의 쿼리를 실행하여 일별 결과를 실제로 저장 ANALYZE ecom.mv_daily_gmv; -- MV의 행 수와 값 분포 통계를 수집하여 실행계획 선택에 활용

원본 쿼리 계획

5. 원본 데이터를 직접 조인·집계하는 실행계획

비교 기준이 되는 원본 쿼리이다.

orders와 order_items를 읽고 Join, Aggregate, Sort를 수행하므로 MV 조피보다 작업량이 많을 수 있다.

탐지할 값: 최상위 Buffers, Planning Time, Execution Time, Join/Aggregate 노드

EXPLAIN (ANALYZE, BUFFERS, SUMMARY)

SELECT date_trunc('day', o.order_ts)::date AS sales_day, -- 주문일을 일 단위로 변환
COUNT(DISTINCT o.order_id) AS order_count, -- 조인 후 중복을 제거한 일별 주문 수
ROUND(SUM(oi.line_total), 2) AS gmv -- 일별 총 상품 판매금액
FROM ecom.orders o -- 원본 주문 테이블 읽기
JOIN ecom.order_items oi -- 원본 주문상품 테이블 읽기
ON oi.order_id = o.order_id -- 주문번호를 기준으로 원본 테이블 두 개 조인
WHERE o.order_status IN ('paid', 'shipped', 'delivered') -- 상태를 필터링한 집계
GROUP BY date_trunc('day', o.order_ts)::date -- 날짜별로 주문 수와 금액 계산
ORDER BY sales_day DESC -- 가장 최근 날짜부터 정렬
LIMIT 30; -- 최근 30개 날짜만 출력

Results 1 X Statistics 1

EXPLAIN (ANALYZE, BUFFERS, SUMMARY) SELECT date_trunc('day', o.or | ^ * Enter a SQL expression to filter results (use Ctrl+Space)

AZ QUERY PLAN

1 Limit (cost=939.49..940.89 rows=30 width=44) (actual time=9.799..10.154 rows=30 loops=1)

2 Buffers: shared hit=140

3 -> GroupAggregate (cost=939.49..1083.25 rows=3068 width=44) (actual time=9.797..10.149 rows=30 loops=1)

4 Group Key: ((date_trunc('day')::text, o.order_ts)::date)

5 Buffers: shared hit=140

6 -> Sort (cost=939.49..960.09 rows=8240 width=18) (actual time=9.771..9.856 rows=1368 loops=1)

7 Sort Key: ((date_trunc('day')::text, o.order_ts)::date) DESC, o.order_id

8 Sort Method: quicksort Memory: 706kB

9 Buffers: shared hit=140

10 -> Hash Join (cost=131.43..403.54 rows=8240 width=18) (actual time=2.372..7.234 rows=8230 loops=1)

11 Hash Cond: (oi.order_id = o.order_id)

12 Buffers: shared hit=140

13 -> Seq Scan on order_items oi (cost=0.00..203.67 rows=10367 width=14) (actual time=0.034..1.046 rows=10367 loops=1)

14 Buffers: shared hit=100

15 -> Hash (cost=93.08..93.08 rows=3068 width=16) (actual time=2.269..2.270 rows=3068 loops=1)

16 Buckets: 4096 Batches: 1 Memory Usage: 176kB

17 Buffers: shared hit=40

18 -> Seq Scan on orders o (cost=0.00..93.08 rows=3068 width=16) (actual time=0.020..1.465 rows=3068 loops=1)

19 Filter: (order_status = ANY ('paid,shipped,delivered')::text[])

20 Rows Removed by Filter: 792

21 Buffers: shared hit=40

22 Planning:

23 Buffers: shared hit=18

24 Planning Time: 0.835 ms

25 Execution Time: 10.257 ms

6. Materialized View 조회 실행계획

이미 계산되어 저장된 sales_day, order_count, gmv만 읽는다.

원본 테이블 조인과 GROUP BY가 사라지는지 원본 실행계획과 비교한다.

탐지할 값: Scan 방식, 최상위 Buffers, Planning Time, Execution Time

EXPLAIN (ANALYZE, BUFFERS, SUMMARY)

SELECT sales_day, -- MV에 저장된 매출 날짜
order_count, -- MV에 저장된 일별 주문 수
gmv -- MV에 저장된 일별 GMV
FROM ecom.mv_daily_gmv -- 원본 조인 대신 사전 집계된 MV만 조회
ORDER BY sales_day DESC -- 가장 최근 날짜부터 정렬
LIMIT 30; -- 최근 30개 날짜만 출력

Results 1 X Statistics 1

EXPLAIN (ANALYZE, BUFFERS, SUMMARY) SELECT sales_day, order_cou | ^ * Enter a SQL expression to filter results (use Ctrl+Space)

AZ QUERY PLAN

1 Limit (cost=0.14..3.26 rows=30 width=20) (actual time=0.014..0.018 rows=30 loops=1)

2 Buffers: shared hit=1 read=1

3 -> Index Scan Backward using ux_mv_daily_gmv_sales_day on mv_daily_gmv (cost=0.14..14.17 rows=135 width=20) (actual time=0.013..0.016 rows=30 loops=1)

4 Buffers: shared hit=1 read=1

5 Planning:

6 Buffers: shared hit=56 read=1

7 Planning Time: 0.582 ms

8 Execution Time: 0.027 ms

MV 조회 결과

7. Materialized View 실제 결과 확인

EXPLAIN 없이 실행하여 구매·재무팀에 전달할 실제 일별 결과를 확인한다.

SELECT sales_day, -- 일별 매출 기준 날짜
order_count, -- 해당 날짜의 중복 없는 판매용 주문 수
gmv -- 해당 날짜의 주문상품 총매출
FROM ecom.mv_daily_gmv -- 미리 저장된 일별 집계 결과 조회
ORDER BY sales_day DESC -- 최근 날짜부터 표시
LIMIT 30; -- 최근 30일분 결과 표시

mv_daily_gmv 1 X Statistics 1

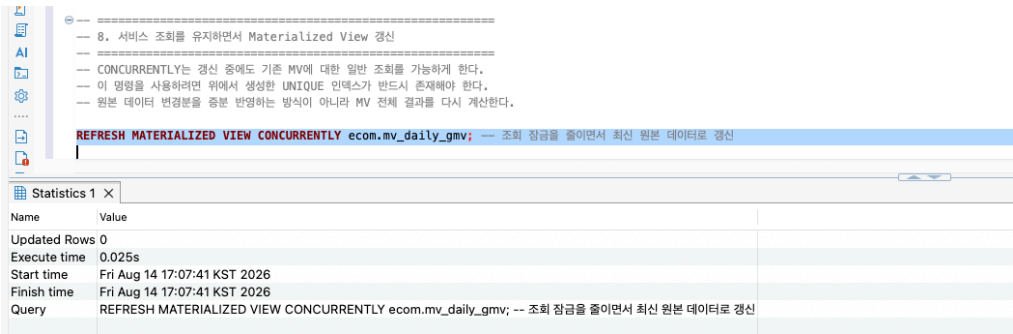
SELECT sales_day, order_count, gmv FROM ecom.mv_daily_gmv ORDER E | ^ * Enter a SQL expression to filter results (use Ctrl+Space)

	sales_day	order_count	gmv
1	2026-09-04	1	234.98
2	2026-09-03	2	577.53
3	2026-08-30	2	492.73
4	2026-08-29	2	716.1
5	2026-08-28	1	339.12
6	2026-08-27	2	470.26
7	2026-08-26	2	493.92
8	2026-08-24	1	171.21
9	2026-08-22	3	400.5
10	2026-08-21	2	647.42
11	2026-08-20	6	1,434.01
12	2026-08-19	3	390.33
13	2026-08-18	2	294.15
14	2026-08-17	2	278.8
15	2026-08-14	21	16,900.68
16	2026-08-13	30	27,756.96
17	2026-08-12	22	18,433.21
18	2026-08-11	31	29,707.93
19	2026-08-10	25	16,932.28
20	2026-08-09	29	25,555.9
21	2026-08-08	30	27,412.01
22	2026-08-07	25	21,950.82
23	2026-08-06	42	39,935.38
24	2026-08-05	27	22,869.32
25	2026-08-04	27	32,913.44
26	2026-08-03	29	31,064.72
27	2026-08-02	35	29,385.72
28	2026-08-01	38	43,562.65
29	2026-07-31	40	43,292.45
30	2026-07-30	29	24,239.39

종합실습4_제출리포트_작성물

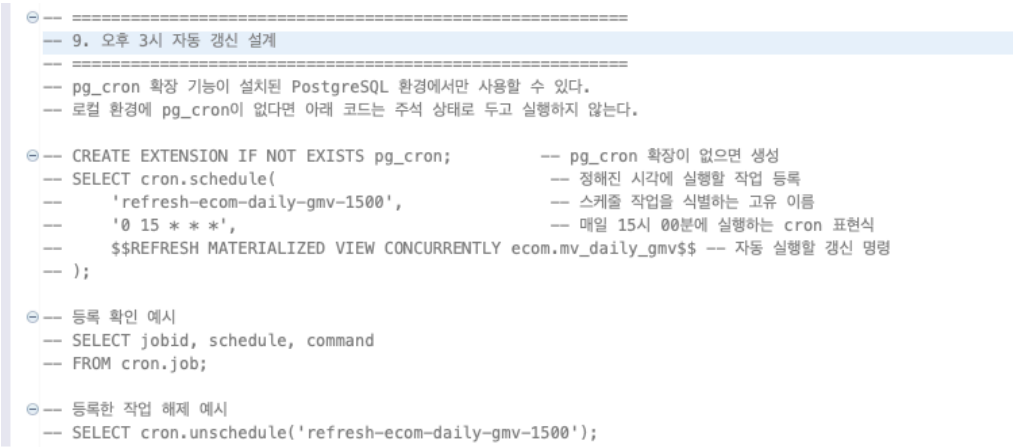
17

MV 갱신



구분	핵심 실행계획	Buffers	Execution Time
원본 JOIN + GROUP BY	_____	_____	_____ ms
<code>mv_daily_gmv</code> 조회	_____	_____	_____ ms

오후 3시 설계



갱신 전략

- 최초 적재: `REFRESH MATERIALIZED VIEW ecom.mv_daily_gmv;`
- 운영 갱신: `REFRESH MATERIALIZED VIEW CONCURRENTLY ecom.mv_daily_gmv;`
- 제안 주기: 매일 오후 3시
- 주의점: 오후 3시 이후 변경사항은 다음 갱신 전까지 조회 결과에 반영되지 않는다.

16. DBMS별 옵티마이저 비교 (Option)

자세한 조사 내용은 `종합실습4_06_DBMS별_옵티마이저_비교.md` 를 참고한다.

DBMS	대표 조인	실행계획 확인	특징
PostgreSQL	Nested Loop, Hash, Merge	<code>EXPLAIN ANALYZE</code>	통계·비용 기반, 확장 통계, GEQO
MySQL	Nested Loop/BKA, Hash	<code>EXPLAIN ANALYZE</code>	optimizer switch·힌트·히스토그램
Oracle	Nested Loop, Hash, Sort Merge	<code>DBMS_XPLAN</code>	Adaptive Plan, SQL Plan Management
SQL Server	Nested Loops, Hash, Merge, Adaptive	Actual Execution Plan	Query Store, Intelligent Query Processing

17. 전체 튜닝 전후 비교

문항	튜닝 전 방식	튜닝 후 방식	전 Buffers	후 Buffers	전 시간	후 시간	감소율
Q1	함수 조건 + 상관 집계	SARG 범위 + 단일 집계	_____	_____	_____	_____	_____%
Q2	상세행 + DISTINCT	주문별 선집계	_____	_____	_____	_____	_____%
Q3	비-SARG 기간 조건	최근 주문 선필터	_____	_____	_____	_____	_____%
Q4	제품별 상관 서브쿼리	전체 1회 집계 + RANK	_____	_____	_____	_____	_____%
Q5	고객별 3회 반복 조회	주문 선집계 + 고객 GROUP BY	_____	_____	_____	_____	_____%
Q6	고객별 EXISTS	Window + BOOL_OR	_____	_____	_____	_____	_____%
Q7	임계치 상관 조회	직접 비교 + 부분 인덱스	_____	_____	_____	_____	_____%
Q8	AVG/COUNT 반복	GROUP BY + HAVING	_____	_____	_____	_____	_____%
Q9	상세행 + DISTINCT	주문별 선집계 + AVG	_____	_____	_____	_____	_____%
Q10	누적/60일 매출 반복 조회	전체 1회 집계 + 정확히 30명	_____	_____	_____	_____	_____%

감소율 계산식:

(튜닝 전 시간 - 튜닝 후 시간) / 튜닝 전 시간 × 100

18. 최종 분석

요청 대응 관점

어떤 주문 상태와 기간을 포함하는지, 주문 수와 주문상품 수를 혼동하지 않았는지, 상위 고객의 선정 기준과 표시 기간이 다른지 먼저 정의해야 한다. 특히 Q10은 최근 60일 매출로 상위 고객을 선정하는 것이 아니라 누적매출로 30명을 선정한 뒤 최근 60일 매출을 계산한다.

튜닝 관점

한계와 추가 확인사항

- 랜덤 시드이므로 재실행 시 결과가 달라진다.
- 로컬 단일 사용자 환경의 실행시간은 운영 부하를 반영하지 않는다.
- 쿠폰 사용 여부와 AOV의 관계는 인과관계가 아니다.
- 인덱스는 조회 성능을 높이지만 INSERT/UPDATE와 저장공간 비용을 추가한다.

최종 결론